

Coding Interview Guide — Volume 4

Advanced Patterns · Java 8+ · with diagrams before the code

Target level: Mid-level (leaning Medium–Hard) software-engineering interviews at companies such as Booking.com, Airbnb, Microsoft, Agoda, UiPath, and organizations of similar range. This volume is the natural step up from Vol 1's twelve core patterns.

What's new in this volume

Volume 1 taught the twelve patterns you must recognize. Volumes 2–3 deepened binary search, tries, DP, graphs and greedy. **Volume 4 is the "next gear"** — the higher-signal techniques that separate a clean Medium solve from a strong one, plus genuinely harder problems that show up in the second coding round.

Every problem is **new** — nothing here repeats Vol 1, 2, or 3.

For every problem you get the same trusted layout you liked in Vol 1:

1. **The idea in plain language** — what the problem is really asking.
2. **A worked example** — concrete input and output.
3. **A diagram first** — a picture of the intuition *before* any code, so you build the mental model before reading Java. Graph and grid problems get **extra** diagrams (adjacency pictures, BFS layers, union-find forests, DP tables).
4. **Brute force in Java** — the obvious baseline.
5. **Optimized in Java** — the solution the interviewer wants, with the trick called out.
6. **Complexity table** — time and space for both versions.

All code targets **Java 8 or later**, compiles as-is, and favors clarity over cleverness.

A note on difficulty

This volume sits in the **Medium** → **Medium-Hard** band. A few problems (Largest Rectangle in Histogram, Longest Increasing Path in a Matrix, Best Time to Buy and Sell Stock IV) are honestly Hard — they're here because the *technique* they teach unlocks a whole family of easier problems. Learn the pattern, not just the problem.

The five advanced patterns in this volume

#	Pattern	When you reach for it
1	Prefix Sums & Difference Arrays	Many range-sum / range-update queries, subarray-sum targets
2	Bit Manipulation & Math	XOR tricks, counting bits, GCD, fast power, digit math
3	Monotonic Stack & Deque	"Next greater/smaller", window max, histogram spans

#	Pattern	When you reach for it
4	Advanced Dynamic Programming	Interval DP, 2D-string DP, stock-series with a state machine
5	Advanced Graphs	Union-Find, cycle detection, multi-source & 0-1 BFS, DAG memoized DFS

How to practice (same discipline as Vol 1)

- **Recognize before you code.** First 60 seconds: *which of these five is it?*
 - **Draw the diagram yourself.** For graph/DP problems, sketch the picture first — exactly as this guide does. If you can draw it, you can code it.
 - **Say the brute force out loud**, then state the optimization and *why* it's correct.
 - **State complexity before you're asked.** It signals seniority.
-

1. Prefix Sums & Difference Arrays

The core idea. A **prefix sum** array P stores the running total: $P[i] = \text{sum of the first } i \text{ elements}$. Once you have it, the sum of *any* range $[l, r]$ is just $P[r+1] - P[l]$ — a single subtraction instead of a loop. The mirror technique, a **difference array**, lets you apply many *range updates* in $O(1)$ each and reconstruct the final array once at the end. Together they turn "sum/update a range" from $O(n)$ per query into $O(1)$.

Reach for it when: you see **many** range-sum queries, a **subarray-sum equals / divisible-by K** target (prefix sum + hash map), or **many range increments** on an array (difference array).

1.1 Subarray Sum Equals K (LC 560)

Problem. Given an integer array `nums` and an integer `k`, return the number of **contiguous** subarrays whose sum equals `k`.

- **Input:** `nums: int[], k: int`. **Output:** the count of contiguous subarrays (`int`) whose elements sum to exactly `k`.
- **Constraints:** $1 \leq \text{nums.length} \leq 2 * 10^4$; $-1000 \leq \text{nums}[i] \leq 1000$; $-10^7 \leq k \leq 10^7$. Elements **can be negative, zero, or repeated** — the array is not sorted.
- **Clarifying assumptions:** confirm the answer can safely exceed `Integer.MAX_VALUE`'s worry range for typical inputs (it won't, given the bounds, but say so); confirm subarrays must be contiguous (not subsequences) and that overlapping subarrays are all counted separately (e.g. `[1,1,1]`, `k=2` counts `[1,1]` twice).
- **Why it's tricky:** negative numbers rule out any two-pointer/sliding-window shrink trick (the window sum isn't monotonic), which is why this needs prefix sums plus a hash map rather than the "shrink from the left" pattern that works for all-positive arrays.

Example.

```
Input:  nums = [1, 2, 3], k = 3
Output: 2      ([1,2] and [3])
```

Intuition (diagram). A subarray sum $[l..r]$ equals $\text{prefix}[r] - \text{prefix}[l-1]$. We want that to equal `k`, i.e. $\text{prefix}[l-1] = \text{prefix}[r] - k$. So as we sweep left to right keeping the running prefix, we ask: *how many earlier prefixes equal `current - k`?* A hash map of "prefix value $\rightarrow$ how many times seen" answers that in $O(1)$.

```

index:      0      1      2
nums:       1      2      3
prefix:  0      1      3      6
          ^

```

empty prefix, seeded in map as {0: 1}

```

i=0 prefix=1 need 1-3=-2 map has -2? no map -> {0:1, 1:1}
i=1 prefix=3 need 3-3= 0 map has 0? yes(1) count+=1 map -> {0:1,1:1,3:1}
i=2 prefix=6 need 6-3= 3 map has 3? yes(1) count+=1 map -> {0:1,1:1,3:1,6:1}

```

count = 2

Approach (step by step)

Key idea / invariant. Maintain a running `prefix` sum and a hash map `seen` of "prefix value $\rightarrow$ number of indices where that value has occurred so far." Before adding the current index to the map, check whether `prefix - k` is already a key: every occurrence of that value is the left boundary of a subarray ending here that sums to exactly `k`.

1. Initialize `seen = {0: 1}` — this represents the "empty prefix" before index 0, which is needed so a subarray starting at index 0 can be found.
2. Initialize `prefix = 0` and `count = 0`.
3. For each element `n` in `nums`: a. Add `n` to `prefix` (this is now `prefix[i]`, the sum of everything up to and including the current element). b. Look up `prefix - k` in `seen`. Add its count (or 0 if absent) to `count` — each hit is one earlier prefix whose "gap" to here sums to `k`. c. Increment `seen[prefix]` by 1 (record that this prefix value has now been seen once more), so later elements can pair with it.
4. Return `count`.

Worked trace (matches the diagram above): after processing all three elements, `count` accumulates 1 at `i=1` (subarray `[1,2]`) and 1 at `i=2` (subarray `[3]`), for a final answer of 2.

Brute force

Try every subarray, sum it.

```

public int subarraySumBrute(int[] nums, int k) {
    int count = 0;
    for (int i = 0; i < nums.length; i++) {
        int sum = 0;
        for (int j = i; j < nums.length; j++) {
            sum += nums[j];
            if (sum == k) count++;
        }
    }
    return count;
}

```

Optimized (prefix sum + hash map)

One pass; the map remembers how often each running prefix has occurred.

```

public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> seen = new HashMap<>();
    seen.put(0, 1); // empty prefix sums to 0, seen once
    int prefix = 0, count = 0;
    for (int n : nums) {
        prefix += n;
        count += seen.getOrDefault(prefix - k, 0); // earlier prefixes that complete a k-
        seen.merge(prefix, 1, Integer::sum);
    }
    return count;
}

```

Version	Time	Space
Brute force	$O(n^2)$	$O(1)$
Prefix + hash map	$O(n)$	$O(n)$

1.2 Contiguous Array (LC 525)

Problem. Given a binary array `nums` (only 0 s and 1 s), return the length of the **longest** contiguous subarray that contains an **equal number of 0 s and 1 s**.

- **Input:** `nums: int[]` with every element either 0 or 1. **Output:** the maximum length (`int`) of a contiguous subarray with equal counts of 0 and 1.
- **Constraints:** $1 \leq \text{nums.length} \leq 10^5$; `nums[i]` is 0 or 1 only.
- **Clarifying assumptions:** confirm that if no such subarray exists (impossible here since `[]` trivially isn't counted but a length-0 answer is valid when nothing balances), the answer is 0; confirm "contiguous" excludes reordering the elements.
- **Why it's tricky:** the natural instinct is to track separate running counts of 0s and 1s and compare them, but the real trick is to **transform the alphabet** (map 0 $\rightarrow$ -1) so that "equal counts" becomes "subarray sums to zero," turning this into the same prefix-sum-collision idea as 1.1, just matching a value against *itself* instead of against `current - k`.

Example.

```

Input:  [0, 1, 0]
Output: 2      ([0,1] or [1,0])

```

Intuition (diagram). Turn every 0 into -1. Now "equal 0s and 1s" means "subarray sums to 0". A subarray `[l..r]` sums to 0 exactly when `prefix[l-1] == prefix[r]`. So the **first** time we see a given prefix value, we record its index; the next time we see the *same* value, the stretch in between nets to zero. Keep the longest such stretch.

index:	-1	0	1	2
nums:		0	1	0
mapped:		-1	+1	-1
prefix:	0	-1	0	-1

prefix 0 first seen at index -1, seen again at index 1 → length $1 - (-1) = 2$ ← best
 prefix -1 first seen at index 0, seen again at index 2 → length $2 - 0 = 2$

Approach (step by step)

Key idea / invariant. Map $0 \rightarrow -1$ and $1 \rightarrow +1$ (conceptually — the code just checks `nums[i] == 1`). Maintain a running `prefix` and a hash map `firstIndex` storing, for each prefix value, the **earliest** index at which it occurred. If the same prefix value reappears at a later index, every element strictly between those two indices nets to zero, i.e. the subarray from just after the first occurrence to the second occurrence has equal 0s and 1s.

1. Initialize `firstIndex = {0: -1}` — prefix 0 "occurs" at virtual index -1, before the array starts, so a balanced subarray beginning at index 0 can still be detected.
2. Initialize `prefix = 0` and `best = 0`.
3. For each index `i` from 0 to `n-1`: a. Update `prefix` by +1 if `nums[i] == 1`, else -1. b. If `prefix` is already a key in `firstIndex`, the subarray from `firstIndex[prefix] + 1` to `i` is balanced; update `best = max(best, i - firstIndex[prefix])`. c. Otherwise, record `firstIndex[prefix] = i` (only the **first** sighting is kept, because we want the longest possible span later).
4. Return `best`.

Worked trace (matches the diagram): at `i=1`, `prefix` returns to 0, which was first seen at index -1, giving length $1 - (-1) = 2$. At `i=2`, `prefix` returns to -1, first seen at index 0, giving length $2 - 0 = 2$. Best stays 2.

Brute force

Check every subarray's balance.

```
public int findMaxLengthBrute(int[] nums) {
    int best = 0;
    for (int i = 0; i < nums.length; i++) {
        int zeros = 0, ones = 0;
        for (int j = i; j < nums.length; j++) {
            if (nums[j] == 0) zeros++; else ones++;
            if (zeros == ones) best = Math.max(best, j - i + 1);
        }
    }
    return best;
}
```

Optimized (prefix sum + first-seen index)

Store the earliest index for each prefix value.

```

public int findMaxLength(int[] nums) {
    Map<Integer, Integer> firstIndex = new HashMap<>();
    firstIndex.put(0, -1); // prefix 0 exists "before" index 0
    int prefix = 0, best = 0;
    for (int i = 0; i < nums.length; i++) {
        prefix += (nums[i] == 1) ? 1 : -1;
        if (firstIndex.containsKey(prefix)) {
            best = Math.max(best, i - firstIndex.get(prefix));
        } else {
            firstIndex.put(prefix, i); // record earliest occurrence only
        }
    }
    return best;
}

```

Version	Time	Space
Brute force	$O(n^2)$	$O(1)$
Prefix + hash map	$O(n)$	$O(n)$

1.3 Subarray Sums Divisible by K (LC 974)

Problem. Given an integer array `nums` and an integer `k`, return the number of (non-empty) contiguous subarrays whose sum is **divisible by `k`**.

- **Input:** `nums: int[]`, `k: int`. **Output:** the count (`int`) of contiguous subarrays whose sum is a multiple of `k` (including 0).
- **Constraints:** $1 \leq \text{nums.length} \leq 3 \times 10^4$; $-10^4 \leq \text{nums}[i] \leq 10^4$; $2 \leq k \leq 10^4$. `nums` can contain negative numbers.
- **Clarifying assumptions:** confirm `k` is always positive and non-zero (so `% k` is well-defined); confirm a subarray summing to exactly 0 counts (it does — 0 is divisible by every `k`).
- **Why it's tricky:** this is 1.1 generalized from "same exact prefix value" to "same prefix value modulo `k`," and the trap is Java's `%` operator returning a **negative** remainder for negative operands (e.g. `-3 % 5 == -3`, not `2`) — you must normalize into `[0, k)` or the bucket lookup silently misses valid pairs.

Example.

```

Input:  nums = [4, 5, 0, -2, -3, 1], k = 5
Output: 7

```

Intuition (diagram). A range sum is divisible by `k` when the two prefix sums bounding it have the **same remainder** mod `k` (because their difference is then a multiple of `k`). So group prefixes by remainder; any two prefixes in the same bucket form a valid subarray. If a remainder has been seen `c` times before, the current prefix pairs with all `c` of them, contributing `c` new subarrays — we add that count as we go (equivalent to summing $C(c, 2)$ over the whole bucket by the end).

```

nums:      4    5    0   -2   -3    1
prefix:  0    4    9    9    7    4    5
mod 5:   0    4    4    4    2    4    0

```

bucket counts start as remainderCount = [1,0,0,0,0] (empty prefix, remainder 0)

step	prefix	mod5	lookup	remainderCount[mod5]	count+=	bucket after
1	4	4	0		+0	[1,0,0,0,1]
2	9	4	1		+1	[1,0,0,0,2]
3	9	4	2		+2	[1,0,0,0,3]
4	7	2	0		+0	[1,0,1,0,3]
5	4	4	3		+3	[1,0,1,0,4]
6	5	0	1		+1	[2,0,1,0,4]

total count = 0+1+2+0+3+1 = 7

negative-remainder fix: ((prefix % k) + k) % k keeps the remainder in [0, k)

Approach (step by step)

Key idea / invariant. Maintain a running prefix sum, normalized mod k into $[0, k)$, and a fixed-size array remainderCount[0..k-1] counting how many earlier prefixes landed in each remainder bucket. Two prefixes with the same remainder differ by a multiple of k , so every pair of prefixes sharing a bucket defines one valid subarray.

1. Initialize remainderCount = new int[k] with remainderCount[0] = 1 (the empty prefix, remainder 0, "exists" before the array starts).
2. Initialize prefix = 0 and count = 0.
3. For each element n in nums: a. Update prefix = ((prefix + n) % k + k) % k — add n , then normalize so the result is always in $[0, k)$ even if the running total is negative. b. Add remainderCount[prefix] to count — every earlier prefix in this same bucket pairs with the current one to form a subarray divisible by k . c. Increment remainderCount[prefix] by 1.
4. Return count.

Worked trace: the table above walks all six elements of nums = [4,5,0,-2,-3,1] with $k=5$; the running count after each step is 0, 1, 3, 3, 6, 7, matching the expected output of 7.

Brute force

Every subarray, test divisibility.

```

public int subarraysDivByKBrute(int[] nums, int k) {
    int count = 0;
    for (int i = 0; i < nums.length; i++) {
        int sum = 0;
        for (int j = i; j < nums.length; j++) {
            sum += nums[j];
            if (sum % k == 0) count++;
        }
    }
    return count;
}

```


Optimized (count prefixes by remainder)

Bucket by normalized remainder; each new prefix pairs with all earlier ones in its bucket.

```
public int subarraysDivByK(int[] nums, int k) {
    int[] remainderCount = new int[k];
    remainderCount[0] = 1; // the empty prefix has remainder 0
    int prefix = 0, count = 0;
    for (int n : nums) {
        prefix = ((prefix + n) % k + k) % k; // normalize into [0, k)
        count += remainderCount[prefix]; // pairs with earlier same-remainder prefixes
        remainderCount[prefix]++;
    }
    return count;
}
```

Version	Time	Space
Brute force	$O(n^2)$	$O(1)$
Prefix remainder buckets	$O(n)$	$O(k)$

1.4 Range Sum Query 2D – Immutable (LC 304)

Problem. Given a fixed 2D matrix, support many queries that return the sum of all elements inside the rectangle with upper-left corner $(r1, c1)$ and lower-right corner $(r2, c2)$, inclusive. The constructor runs once; `sumRegion` may be called many times.

- **Input:** constructor takes `matrix: int[][]`; `sumRegion(r1, c1, r2, c2)` takes four `int` s.
Output: `sumRegion` returns the rectangle's sum (`int`).
- **Constraints:** $1 \leq \text{rows}, \text{cols} \leq 200$; $-10^5 \leq \text{matrix}[i][j] \leq 10^5$; $0 \leq r1 \leq r2 < \text{rows}$; $0 \leq c1 \leq c2 < \text{cols}$; up to 10^4 calls to `sumRegion`.
- **Clarifying assumptions:** confirm the matrix is **immutable** after construction (no cell updates — that variant, LC 308, needs a Fenwick tree/BIT instead); confirm indices are always valid (no bounds checking required).
- **Why it's tricky:** the 1D "subtract two prefixes" trick doesn't directly generalize — naively doing `P[r2][c2] - P[r1][c1]` double-subtracts nothing and under-subtracts the two strips outside the rectangle; you need **inclusion-exclusion** across both dimensions to correctly cancel the doubly-subtracted corner exactly once.

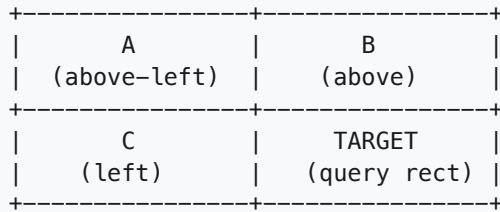
Example.

```
matrix:
3 0 1 4 2
5 6 3 2 1
1 2 0 1 5
4 1 0 1 7
1 0 3 0 5

sumRegion(2,1,4,3) = 8
```

Intuition (diagram). Build a 2D prefix $P[r][c]$ = sum of the whole rectangle from the top-left corner $(0,0)$ to $(r-1, c-1)$. Any sub-rectangle is then the big block down to its bottom-right corner, minus the strip above it, minus the strip to its left, **plus** the top-left corner region that got subtracted twice — classic inclusion–exclusion.

Full block down to $(r2, c2)$, split into four regions relative to the query rectangle:



$P[r1][c1]$ = A (top-left corner block)
 $P[r1][c2+1]$ = A + B (everything above TARGET, including A)
 $P[r2+1][c1]$ = A + C (everything left of TARGET, including A)
 $P[r2+1][c2+1]$ = A + B + C + TARGET (the full block)

$TARGET = (A+B+C+TARGET) - (A+B) - (A+C) + A$
 $= P[r2+1][c2+1] - P[r1][c2+1] - P[r2+1][c1] + P[r1][c1]$
 $\quad \quad \quad \wedge \text{ full block} \quad \quad \wedge \text{ subtract B+A} \quad \wedge \text{ subtract C+A} \quad \wedge \text{ add A back (removed twice)}$

Approach (step by step)

Key idea / invariant. Precompute $p[r][c]$ = sum of the sub-matrix from $(0,0)$ to $(r-1, c-1)$ inclusive, using a $(rows+1) \times (cols+1)$ array padded with a zero row/column so no boundary case needs special-casing. Each cell is filled from three neighbors already computed, mirroring the 1D recurrence extended to two dimensions.

- Build phase — recurrence:** $p[r+1][c+1] = matrix[r][c] + p[r][c+1] + p[r+1][c] - p[r][c]$. In words: the prefix sum up to (r, c) is the current cell, plus everything above it, plus everything to its left, minus the top-left overlap counted twice.
- Fill order: row by row (or column by column), since $p[r+1][c+1]$ depends only on $p[r][c+1]$ (already-filled previous row) and $p[r+1][c]$ (already-filled earlier column in the current row).
- Query phase:** for `sumRegion(r1, c1, r2, c2)`, apply the inclusion–exclusion formula derived in the diagram: $p[r2+1][c2+1] - p[r1][c2+1] - p[r2+1][c1] + p[r1][c1]$.
- Every query is now 4 array reads and 3 additions — $O(1)$ regardless of rectangle size.

Worked trace: for `sumRegion(2,1,4,3)` on the example matrix, $p[5][4]$ (full block down to row 4, col 3) minus $p[2][4]$ (rows above row 2) minus $p[5][1]$ (columns left of col 1) plus $p[2][1]$ (the double-subtracted corner) evaluates to 8, matching the expected output.

Brute force

Sum the rectangle on every query.

```

class NumMatrixBrute {
    private final int[][] m;
    public NumMatrixBrute(int[][] matrix) { this.m = matrix; }

    public int sumRegion(int r1, int c1, int r2, int c2) {
        int sum = 0;
        for (int r = r1; r <= r2; r++)
            for (int c = c1; c <= c2; c++)
                sum += m[r][c];
        return sum;
    }
}

```

Optimized (2D prefix sum, inclusion–exclusion)

Precompute once; each query is four array lookups.

```

class NumMatrix {
    private final int[][] p;          // p[r][c] = sum of matrix[0..r-1][0..c-1]

    public NumMatrix(int[][] matrix) {
        int rows = matrix.length, cols = matrix[0].length;
        p = new int[rows + 1][cols + 1];
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++)
                p[r + 1][c + 1] = matrix[r][c]
                    + p[r][c + 1] + p[r + 1][c] - p[r][c];
    }

    public int sumRegion(int r1, int c1, int r2, int c2) {
        return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1];
    }
}

```

Version	Time	Space
Brute force	$O(1)$ build, $O(\text{rows} \cdot \text{cols})$ per query	$O(1)$
2D prefix sum	$O(\text{rows} \cdot \text{cols})$ build, $O(1)$ per query	$O(\text{rows} \cdot \text{cols})$

1.5 Corporate Flight Bookings (LC 1109)

Problem. There are n flights labeled $1..n$. Each booking $[\text{first}, \text{last}, \text{seats}]$ adds seats to every flight from first to last inclusive. Given all bookings, return an array where index i holds the total seats booked for flight $i+1$.

- Input:** `bookings: int[][]` where each row is `[first, last, seats]` (1-indexed flight numbers), and `n: int` (total number of flights). **Output:** `int[]` of length `n`, the total seats per flight.
- Constraints:** $1 \leq n \leq 2 \cdot 10^4$; $1 \leq \text{bookings.length} \leq 2 \cdot 10^4$; `bookings[i].length == 3`; $1 \leq \text{first}_i \leq \text{last}_i \leq n$; $1 \leq \text{seats}_i \leq 10^4$.

- **Clarifying assumptions:** confirm `first` and `last` are both inclusive and 1-indexed; confirm bookings may overlap arbitrarily (they do, and all overlapping bookings simply add).
- **Why it's tricky:** applying each booking directly is $O(\text{range})$ per booking, which is fine until ranges and booking counts are both large; the trick is to stop touching every element in the range and instead touch only its **two boundaries**, deferring the actual "spread" to a single $O(n)$ pass at the end — this is the difference-array pattern, the mirror image of a prefix sum.

Example.

```
Input: bookings = [[1,2,10],[2,3,20],[2,5,25]], n = 5
Output: [10, 55, 45, 25, 25]
```

Intuition (diagram — difference array). Instead of adding `seats` to every position in a range ($O(\text{range})$ each), record only the **edges** of the change: `+seats` at `first`, and `-seats` just after `last`. After all bookings, a single running prefix sum "spreads" each increment across its range. This is the difference array — the inverse of a prefix sum.

```
flight index (0-indexed):      0      1      2      3      4      5(extra)
booking [1,2,10] -> diff[0]+=10, diff[2]-=10
booking [2,3,20] -> diff[1]+=20, diff[3]-=20
booking [2,5,25] -> diff[1]+=25, diff[5]-=25

diff:                          +10     +45     -10     -20      0     -25
                               ^         ^
                               |         +10(b1) +20(b2) +25(b3) = 45
                               +10 from booking 1

running sum (sweep left to right, drop the extra slot):
i=0: running=10          -> answer[0] = 10
i=1: running=10+45=55    -> answer[1] = 55
i=2: running=55-10=45    -> answer[2] = 45
i=3: running=45-20=25    -> answer[3] = 25
i=4: running=25+0=25     -> answer[4] = 25

answer = [10, 55, 45, 25, 25]
```

Approach (step by step)

Key idea / invariant. A difference array `diff` of size `n+1` stores, at each index, how much the *value changes* relative to the previous index — not the value itself. Adding `seats` to a whole range `[first, last]` then costs $O(1)$: bump `diff[first]` up by `seats` (the increase starts here) and bump `diff[last+1]` down by `seats` (the increase ends after `last`). A final prefix-sum sweep over `diff` reconstructs the real per-flight totals, because summing the "changes" up to index `i` gives the value at index `i`.

1. Initialize `diff = new int[n + 1]` (the extra slot guarantees `last + 1` is always a valid index, even when `last == n`).
2. For each booking `[first, last, seats]` (convert to 0-indexed): `diff[first] += seats` and `diff[last+1] -= seats`.
3. Sweep once: maintain `running = 0`; for `i` from `0` to `n-1`, do `running += diff[i]` and set `answer[i] = running`. This converts "boundary deltas" back into "actual seats at each flight,"

exactly like turning a difference array back into its original array.

4. Return `answer` .

Worked trace: matches the diagram — after folding in all three bookings' boundary marks, the diff array is `[10, 45, -10, -20, 0, -25]` ; sweeping it produces running totals `10, 55, 45, 25, 25` , which is the expected output.

Brute force

Apply each booking element by element.

```
public int[] corpFlightBookingsBrute(int[][] bookings, int n) {
    int[] answer = new int[n];
    for (int[] b : bookings) {
        for (int flight = b[0]; flight <= b[1]; flight++) {
            answer[flight - 1] += b[2];          // 1-indexed flights
        }
    }
    return answer;
}
```

Optimized (difference array)

Mark boundaries in $O(1)$ per booking, then one prefix-sum pass.

```
public int[] corpFlightBookings(int[][] bookings, int n) {
    int[] diff = new int[n + 1];          // extra slot so "last+1" is always in range
    for (int[] b : bookings) {
        diff[b[0] - 1] += b[2];           // start of the range (0-indexed)
        diff[b[1]] -= b[2];              // one past the end
    }
    int[] answer = new int[n];
    int running = 0;
    for (int i = 0; i < n; i++) {
        running += diff[i];
        answer[i] = running;
    }
    return answer;
}
```

Version	Time	Space
Brute force	$O(\text{bookings} \cdot \text{range})$	$O(1)$
Difference array	$O(\text{bookings} + n)$	$O(n)$

2. Bit Manipulation & Math

The core idea. Numbers are bit patterns, and a handful of bit tricks turn "loop over everything" into $O(1)$ or $O(\log n)$. The three you must know cold: **XOR** ($a \oplus a = 0$, $a \oplus 0 = a$, and XOR is order-independent — perfect for cancelling pairs); $n \& (n-1)$ (clears the lowest set bit); and $n \& -n$ (isolates the lowest set bit). On the math side, **GCD (Euclid)** and **fast exponentiation** (square-and-multiply) show up constantly.

Reach for it when: you see "every element appears twice except...", counting set bits, "without using $+$ / $*$ /division", powers, GCD/LCM, or digit/base conversions.

2.1 Single Number II (LC 137)

Problem. Given an integer array `nums` in which **every element appears exactly three times** except for one element, which appears exactly once, find and return that single element. You must do it in $O(n)$ time and, ideally, $O(1)$ extra space.

- **Input:** `nums` — an array of 32-bit signed integers.
- **Output:** the one integer that appears exactly once.
- **Constraints:** $1 \leq \text{nums.length} \leq 3 \times 10^4$; each value fits in a 32-bit signed int (can be negative); every value except one appears exactly three times.
- **Clarifying assumptions:** exactly one element violates the "appears three times" rule (the input is guaranteed valid); a hash map is fine for a first pass, but the interviewer is looking for the $O(1)$ -space bit trick.

Why it's tricky: the usual XOR-cancels-pairs trick (from *Single Number I*) only works when duplicates come in **pairs** — XOR-ing a value with itself three times leaves the value itself, not zero, so it doesn't cancel. You need a mechanism that cancels multiples of **three**, which means reasoning about each bit position separately rather than the whole number at once.

Example.

```
Input:  [2, 2, 3, 2]
Output: 3
```

Intuition (diagram). Look at each bit position independently. If every number appeared three times, each column of bits would sum to a multiple of 3. The one lonely number is exactly the leftover — the bits where the column count is **not** divisible by 3.

numbers in binary (bit columns, 3 bits shown):

	bit2	bit1	bit0	
2 =	0	1	0	
2 =	0	1	0	
3 =	0	1	1	
2 =	0	1	0	
	----	----	----	
col sum:	0	4	1	
mod 3:	0	1	1	-> binary 011 = 3 (the single number)

Approach (step by step)

Key idea: process the 32 bit positions independently. For each bit position, count how many numbers in the array have that bit set. Because every number except one appears three times, that count is a multiple of 3 **plus 0 or 1** depending on whether the single number has that bit set. Taking `count % 3` recovers exactly the single number's bit.

1. Initialize `result = 0`.
2. For each bit position `bit` from 0 to 31: a. Walk every number in `nums` and sum up `(n >> bit) & 1` — this counts how many numbers have a 1 in that position. b. If that sum is **not divisible by 3**, the single number must have a 1 in this position, so set bit `bit` in `result`.
3. After all 32 positions are processed, `result` holds the single number.

Worked trace on [2, 2, 3, 2] (using 3 bits: 010, 010, 011, 010): - bit0: bits are 0,0,1,0 → sum = 1 → `1 % 3 = 1` → set bit0 of result. - bit1: bits are 1,1,1,1 → sum = 4 → `4 % 3 = 1` → set bit1 of result. - bit2: bits are 0,0,0,0 → sum = 0 → `0 % 3 = 0` → leave bit2 clear. - `result = 011` binary = 3. Matches the expected output.

Brute force

Count occurrences with a hash map.

```
public int singleNumberBrute(int[] nums) {
    Map<Integer, Integer> count = new HashMap<>();
    for (int n : nums) count.merge(n, 1, Integer::sum);
    for (Map.Entry<Integer, Integer> e : count.entrySet())
        if (e.getValue() == 1) return e.getKey();
    return -1;
}
```

Optimized (bit-column counting, O(1) space)

For each of the 32 bit positions, sum the bits and take mod 3.

```

public int singleNumber(int[] nums) {
    int result = 0;
    for (int bit = 0; bit < 32; bit++) {
        int sum = 0;
        for (int n : nums) {
            sum += (n >> bit) & 1;    // count how many numbers have this bit set
        }
        if (sum % 3 != 0) {
            result |= (1 << bit);    // the single number owns this bit
        }
    }
    return result;
}

```

Version	Time	Space
Hash map	$O(n)$	$O(n)$
Bit counting	$O(32 \cdot n) = O(n)$	$O(1)$

2.2 Counting Bits (LC 338)

Problem. Given a non-negative integer n , return an array `ans` of length $n + 1$ where `ans[i]` is the number of `1` bits in the binary representation of `i`, for every `i` from `0` to `n`. Aim for $O(n)$ total time (not $O(n \log n)$ from counting each number's bits from scratch).

- **Input:** a single non-negative integer n .
- **Output:** an integer array of length $n + 1$.
- **Constraints:** $0 \leq n \leq 10^5$ (typical LeetCode bound).
- **Clarifying assumptions:** $n = 0$ should return `[0]`; the algorithm should ideally avoid calling a built-in popcount function since that's considered "cheating" the $O(n)$ DP intent.

Why it's tricky: the naive approach (call `Integer.bitCount` or loop over bits for every `i`) is $O(n \log n)$ and treats every number independently. The insight is that each number's bit count can be derived in $O(1)$ from a **smaller number's already-computed answer**, turning this into a DP over integers rather than 32 independent bit scans.

Example.

```

Input:  n = 5
Output: [0, 1, 1, 2, 1, 2]    (0,1,10,11,100,101)

```

Intuition (diagram). `i & (i-1)` removes the lowest set bit, giving a **smaller** number whose answer we already computed. So `bits[i] = bits[i & (i-1)] + 1`. Each number is built from a previously solved one in $O(1)$ — classic DP over bits.

```

i          = 6 = 110
i - 1      = 5 = 101
i & (i-1)  =   4 = 100  ->  bits[6] = bits[4] + 1 = 1 + 1 = 2
(we dropped exactly one set bit, so add one back)

```


Approach (step by step)

Key idea / recurrence: $dp[i] = dp[i \& (i-1)] + 1$. In words: $i \& (i-1)$ clears the lowest set bit of i , producing a strictly smaller number that has exactly one fewer 1 bit than i . Since that smaller number's bit count is already computed (we fill left to right), we just add 1 back for the bit we cleared.

- **State meaning:** $dp[i]$ = number of set bits in i .
- **Base case:** $dp[0] = 0$ (zero has no set bits).
- **Fill order:** ascending, from $i = 1$ to n , because $i \& (i-1)$ is always strictly less than i , so its answer is guaranteed to already be filled in.

An equivalent, commonly taught recurrence for the same DP is $dp[i] = dp[i \gg 1] + (i \& 1)$: shift i right by one (dropping the lowest bit) and add back 1 if that dropped bit was a 1. Both recurrences are $O(1)$ per element and $O(n)$ overall; the code below uses the $i \& (i-1)$ form.

1. Allocate `bits[0..n]`, with `bits[0] = 0`.
2. For i from 1 to n : look up `bits[i & (i-1)]` (already computed, smaller index) and add 1.
3. Return `bits`.

Worked trace for $n = 5$:

i	binary	$i \& (i-1)$	$bits[i \& (i-1)]$	$bits[i]$
0	000	—	—	0
1	001	000	0	1
2	010	000	0	2
3	011	010	2	3
4	100	000	0	4
5	101	100	4	5

Result: `[0, 1, 1, 2, 1, 2]` — matches the expected output.

Brute force

Count bits of each number with `Integer.bitCount` (or a per-number loop).

```
public int[] countBitsBrute(int n) {
    int[] result = new int[n + 1];
    for (int i = 0; i <= n; i++) {
        result[i] = Integer.bitCount(i); // 0(number of bits) each
    }
    return result;
}
```

Optimized (DP with drop-lowest-bit)

Each answer reuses a smaller, already-computed one.

```
public int[] countBits(int n) {
    int[] bits = new int[n + 1];
    for (int i = 1; i <= n; i++) {
        bits[i] = bits[i & (i - 1)] + 1; // one more set bit than i with its lowest bit
    }
    return bits;
}
```

Version	Time	Space
Per-number count	$O(n \cdot \log n)$	$O(n)$

Version	Time	Space
DP (drop lowest bit)	O(n)	O(n)

2.3 Sum of Two Integers (LC 371)

Problem. Given two integers `a` and `b`, return their sum, but you may **not** use the `+` or `-` operators anywhere in your solution.

- **Input:** two 32-bit signed integers `a`, `b`.
- **Output:** the integer `a + b`.
- **Constraints:** `-1000 <= a, b <= 1000` (typical LeetCode bound); either value may be negative or zero.
- **Clarifying assumptions:** bitwise operators (`^`, `&`, `<<`, `>>`), comparisons, and loops are all allowed — only arithmetic `+` / `-` are off the table; assume standard 32-bit two's-complement overflow semantics (Java `int` wraps around, which is fine here).

Why it's tricky: you have to re-derive what "addition" actually means at the bit level. The trick is splitting addition into two independent operations that combine only bitwise-safe operators: a **carry-less sum** (XOR) and a **carry** (AND, shifted), then repeatedly feeding the carry back in until there is none left — exactly how a full-adder circuit works in hardware.

Example.

```
Input:  a = 2, b = 3
Output: 5
```

Intuition (diagram). Binary addition splits into two parts: the **sum without carry** is `a ^ b` (XOR adds bit-by-bit but forgets carries), and the **carry** is `(a & b) << 1` (a carry happens only where both bits are 1, and it lands one position left). Repeat until there's no carry left.

```
a = 2 = 010
b = 3 = 011
XOR (no-carry sum): 001
AND<<1 (carry):    100
now add 001 + 100: XOR = 101, carry = 000 -> carry is 0 -> done -> 101 = 5
```

Approach (step by step)

Key idea: for any two bits, their sum without carrying is XOR, and a carry is produced (into the next position left) exactly when **both** bits are 1, i.e. AND shifted left by one. Adding two numbers is then: compute the no-carry sum, compute the carry, and **add the carry back in** — which is itself another instance of the same problem, so you loop.

Invariant: at the start of each iteration, the *true* sum of the original `a` and `b` equals the *true* sum of the current `a` and `b`. Each iteration moves all carrying into `b` and strips it out of `a`, so `b` shrinks toward zero.

1. While `b != 0` (there is still a carry to fold in): a. Compute `carry = (a & b) << 1` — the bits that will carry into the next position. b. Compute the new `a = a ^ b` — the sum of the current `a` and `b`, ignoring carries. c. Set `b = carry` — the leftover carry becomes the new "amount left to add."
2. When `b` reaches `0`, there is nothing left to carry, and `a` holds the final sum.

Worked trace on `a = 2 (010)`, `b = 3 (011)`: iteration | `a` | `b` (carry-in) | `a & b` | `carry = (a&b)<<1` | new `a = a^b` | new `b` | |-----|-----|-----|-----|-----|-----|-----| | start | 010 (2) | 011 (3) | 010 | 001 | 100 | 001 | 2 | 001 (1) | 100 (4) | 000 | 000 | 101 | 000 |

`b` is now `0`, loop ends, return `a = 101 = 5`. Matches the expected output.

Brute force

The "obvious" version literally uses `+` — which is banned — so there is no meaningful brute force here. The bit approach *is* the intended solution; a naive fallback would be incrementing in a loop (`while (b-- > 0) a++`), which is $O(b)$ and also uses `+/−`.

Optimized (XOR sum + carry loop)

Loop until the carry is exhausted.

```
public int getSum(int a, int b) {
    while (b != 0) {
        int carry = (a & b) << 1;    // where both bits are 1, carry shifts left
        a = a ^ b;                  // add without carrying
        b = carry;                  // now fold the carry back in
    }
    return a;
}
```

Version	Time	Space
Increment loop	$O(b)$	$O(1)$
XOR + carry	$O(1) (\leq 32 \text{ iterations})$	$O(1)$

2.4 Pow(x, n) (LC 50)

Problem. Implement `pow(x, n)`, which calculates `x` raised to the power `n` (i.e. `x^n`), where `n` is a signed integer that **can be negative or zero**. Aim for $O(\log n)$ time.

- **Input:** a `double x` and an `int n`.
- **Output:** a `double` equal to `x^n`.
- **Constraints:** $-100.0 < x < 100.0$; $-2^{31} \leq n \leq 2^{31} - 1$; `x^n` is guaranteed not to overflow or underflow to an unrepresentable value for the test cases used.
- **Clarifying assumptions:** `n = 0` should return `1.0` (including `x = 0`, by convention); negative `n` means `x^n = 1 / x^(-n)`; watch out for `n = Integer.MIN_VALUE`, whose negation overflows a 32-bit `int`, so it must be widened (e.g. to `long`) before negating.

Why it's tricky: the naive loop multiplying `x` by itself `n` times is $O(n)$ and also mishandles negative exponents and the `Integer.MIN_VALUE` overflow edge case. The $O(\log n)$ trick requires seeing that repeated squaring lets you skip most multiplications, guided by the **binary digits of the exponent** — the same idea as base-2 fast exponentiation used in modular arithmetic.

Example.

```
Input:  x = 2.0, n = 10
Output: 1024.0
```

Intuition (diagram). Don't multiply `x` by itself `n` times. **Square-and-multiply:** $x^{10} = (x^5)^2$, and $x^5 = x \cdot (x^2)^2$. Reading `n` in binary, each bit tells you whether to fold the current running square into the answer. That's $O(\log n)$ multiplications.

```
n = 10 = 1010 (binary)
base = x
bit0 = 0 -> skip;      base = x^2
bit1 = 1 -> answer *= x^2; base = x^4
bit2 = 0 -> skip;      base = x^8
bit3 = 1 -> answer *= x^8
answer = x^2 * x^8 = x^10
```

Approach (step by step)

Key idea: write the exponent `n` in binary. x^n is then the product of $x^{(2^k)}$ for every bit position `k` where `n` has a 1 bit — because $n = \text{sum of } 2^k \text{ over set bits}$, and $x^{(\text{sum})} = \text{product of } x^{(2^k)}$. We build the powers-of-two powers of `x` by repeated squaring (`base = base * base` doubles the exponent each time), and fold `base` into the running `answer` exactly when the corresponding bit of `n` is 1.

Invariant: at the top of each loop iteration, `base` holds $x^{(2^k)}$ for the current bit position `k`, and `answer` holds the product of all $x^{(2^j)}$ for lower bits `j` that were already set to 1.

1. If `n` is negative, replace `x` with `1/x` and `n` with `-n` (widen to `long` first so negating `Integer.MIN_VALUE` doesn't overflow).
2. Initialize `answer = 1.0`, `base = x`.
3. While the (now non-negative) exponent `p > 0`: a. If the lowest bit of `p` is 1 (`p & 1 == 1`), multiply it into the answer: `answer *= base`. b. Square the base: `base *= base` (this is now $x^{(2^{(k+1)})}$, ready for the next bit). c. Shift the exponent right one bit: `p >>= 1`.
4. Return `answer`.

Worked trace on `x = 2.0`, `n = 10` (binary 1010):

step	p (binary)	lowest bit	action	base after	answer after
1	1010	0	skip multiply	$x^2 = 4$	1
2	0101	1	multiply	$x^4 = 16$	$x^2 = 4$
3	0010	0	skip multiply	$x^8 = 256$	$4 \cdot x^8 = 1024$
4	0001	1	multiply	$x^{16} = 65536$	$1024 \cdot x^{16} = 1024$
5	0000	—	loop ends (<code>p == 0</code>)	—	1024

`answer = 1024.0`, matching 2^{10} .

Brute force

Multiply `n` times.

```
public double myPowBrute(double x, int n) {
    double result = 1.0;
    long p = Math.abs((long) n);          // cast to long: -2^31 has no positive int
    for (long i = 0; i < p; i++) result *= x;
    return n < 0 ? 1.0 / result : result;
}
```

Optimized (fast exponentiation)

Square the base, multiply in the bits that are set.

```
public double myPow(double x, int n) {
    long p = n;                          // widen so we can negate INT_MIN safely
    if (p < 0) { x = 1 / x; p = -p; }
    double result = 1.0, base = x;
    while (p > 0) {
        if ((p & 1) == 1) result *= base; // this binary digit of the exponent is set
        base *= base;                    // square for the next bit
        p >>= 1;
    }
    return result;
}
```

Version	Time	Space
Multiply <code>n</code> times	$O(n)$	$O(1)$
Fast exponentiation	$O(\log n)$	$O(1)$

2.5 Excel Sheet Column Number (LC 171)

Problem. Given a string `columnTitle` that represents an Excel-style column title ("A"→1, "B"→2, ... "Z"→26, "AA"→27, "AB"→28, ...), return its corresponding column number.

- **Input:** a string of uppercase letters A - Z .
- **Output:** an integer, the 1-indexed column number.
- **Constraints:** $1 \leq \text{columnTitle.length} \leq 7$; all characters are uppercase English letters; the result fits in a 32-bit signed integer (LeetCode caps `columnTitle` so the answer stays within `int` range).
- **Clarifying assumptions:** the input is always well-formed (non-empty, uppercase only, no leading/trailing whitespace); this is **bijective base-26**, not standard base-26 — there is no digit that represents zero, which is why A maps to 1 rather than 0 .

Why it's tricky: it looks like ordinary base conversion, but standard positional systems have a 0 digit and k digits represent numbers $0 \dots k^n - 1$. Here A represents 1, not 0, so naively treating this as base-26 with digits $0 \dots 25$ would be off — the fix is simply letting each letter contribute `charAt - 'A' +`

1 (i.e. 1..26) as its place value, which turns out to handle the "no zero" quirk automatically once you fold digits left-to-right with Horner's rule.

Example.

```
Input:  "AB"
Output: 28          (A=1 -> 1*26 + B=2 -> 28)
```

Intuition (diagram). Read left to right like any positional number: each new letter multiplies the running total by 26 and adds the letter's value (A=1..Z=26). The "no zero" part is handled naturally because A maps to 1, not 0.

```
"AB"
start result = 0
'A' -> result = 0*26 + 1 = 1
'B' -> result = 1*26 + 2 = 28
```

Approach (step by step)

Key idea: this is Horner's rule for evaluating a positional number left to right: to fold in a new digit `d`, multiply the running result by the base and add `d`. Because each letter maps to 1..26 (never 0), the running total is automatically consistent with the "bijective base-26" scheme without any special-casing.

1. Initialize `result = 0`.
2. For each character `c` in `columnName`, left to right: a. Compute its digit value `digit = c - 'A' + 1` (so `A` $\rightarrow$ 1, ..., `Z` $\rightarrow$ 26). b. Fold it in: `result = result * 26 + digit`.
3. After the last character, `result` is the column number.

Worked trace on "AB" :- start: `result = 0`. - 'A': `digit = 1`; `result = 0*26 + 1 = 1`. - 'B': `digit = 2`; `result = 1*26 + 2 = 28`. - Final `result = 28`, matching the expected output.

Brute force

Compute each place value as an explicit power of 26.

```
public int titleToNumberBrute(String s) {
    int result = 0, power = 0;
    for (int i = s.length() - 1; i >= 0; i--) {
        int digit = s.charAt(i) - 'A' + 1;
        result += digit * (int) Math.pow(26, power);    // explicit place value
        power++;
    }
    return result;
}
```

Optimized (Horner's rule, left to right)

Fold in each digit with a single multiply-add — no `pow`, no rounding risk.

```
public int titleToNumber(String s) {  
    int result = 0;  
    for (int i = 0; i < s.length(); i++) {  
        int digit = s.charAt(i) - 'A' + 1;    // A=1, ..., Z=26  
        result = result * 26 + digit;  
    }  
    return result;  
}
```

Version	Time	Space
Explicit powers	$O(n)$ with <code>pow</code> overhead	$O(1)$
Horner's rule	$O(n)$	$O(1)$

3. Monotonic Stack & Deque

The core idea. A **monotonic stack** keeps its elements sorted (increasing or decreasing) as you push. When a new element would break the order, you **pop** everything it dominates — and each pop is the moment you've *found the answer* for the popped element (its "next greater" or "next smaller"). Because every element is pushed and popped at most once, the whole scan is $O(n)$ even though it feels like nested loops. A **monotonic deque** is the sliding-window cousin: it maintains the window's max/min in $O(1)$ amortized.

Reach for it when: you see "next greater / next smaller element", "span until a bigger value", histogram/area problems, or "max/min of every window of size k ".

3.1 Next Greater Element II (LC 503)

Problem. You are given a **circular** integer array `nums` (the last element wraps around to the first). For every element, return the **next greater element** — the first element you encounter when scanning forward (wrapping once if needed) that is strictly larger. If no such element exists, its answer is `-1`. Return the answers as an array in the same order as `nums`.

- **Constraints:** `1 <= nums.length <= 10^4`; `-10^9 <= nums[i] <= 10^9`.
- **Clarifying assumptions:** duplicates are allowed ("greater" means strictly greater, not greater-or-equal); each element wraps around **at most once** (you never need to scan more than `n - 1` steps ahead of any starting index).
- **Why it's tricky:** the circularity makes it tempting to physically duplicate the array; the clean trick is to iterate `2n` times over `i % n` while only *pushing* indices during the first pass — the second pass exists purely to resolve leftover elements.

Example.

```
Input:  [1, 2, 1]
Output: [2, -1, 2]    (the last 1 wraps around to the leading 2)
```

Intuition (diagram). Keep a stack of **indices** whose "next greater" is still unknown. Because we only ever pop an index when a *bigger* value shows up, the values at the indices still on the stack are always in **decreasing order** from bottom to top — that's the monotonic invariant. When a bigger value arrives, it resolves everyone smaller sitting on top of the stack: pop them and record the current value as their answer. To handle the circular wrap, iterate `2n` times using `i % n`, but only push indices seen in the first `n` steps (indices from the second pass have already had their real "one lap forward" chance).

Approach (step by step)

1. **Invariant:** the stack holds indices `i0 < i1 < ... < ik` (in the order pushed) such that `nums[i0] > nums[i1] > ... > nums[ik]` — strictly decreasing values, top = most recently pushed = smallest so far without a resolved answer.

2. **Why each index is pushed and popped at most once (the $O(n)$ argument):** every index is pushed exactly once (guarded by `if (i < n) stack.push(i)`), and once popped it is never pushed again. Since the total number of pushes over the whole run is bounded by `n`, the total number of pops is also bounded by `n` — so even though the `while` loop can pop several elements at one `i`, the *sum* of all pop-work across the whole run is $O(n)$, not $O(n)$ per step. That's the amortized $O(n)$ for $2n$ total steps.
3. Walk `i` from `0` to `2n - 1`; let `cur = nums[i % n]`.
4. While the stack is non-empty and `nums[stack.peek()] < cur`: pop the index and set its answer to `cur` (we just found its next greater element).
5. If `i < n` (still in the first lap), push `i` — indices from the second lap are only used to *resolve* leftovers, never re-pushed.
6. Any index still on the stack after the full $2n$ pass never found a greater element, so it keeps its default answer of `-1`.

Worked trace on `nums = [1, 2, 1]` (`n = 3`, so `i` runs `0..5`):

i	cur	stack (before)	action
0	1	[]	push idx 0 → [0]
1	2	[0]	nums[0]=1 < 2: pop 0, ans[0]=2 → []; push 1 → [1]
2	1	[1]	nums[1]=2, not < 1: stop → push 2 → [1,2]
3	1	[1,2]	wrap: nums[2]=1, not < 1: stop (no push, i>=n)
4	2	[1,2]	wrap: nums[2]=1 < 2: pop 2, ans[2]=2 → [1]; nums[1]=2 not < 2
5	1	[1]	wrap: nums[1]=2, not < 1: stop (no push, i>=n)

Index `1` is never popped, so `ans[1] = -1`. Final answer: `[2, -1, 2]`.

Brute force

For each index, scan forward up to `n` steps (with wrap).

```
public int[] nextGreaterElementsBrute(int[] nums) {
    int n = nums.length;
    int[] ans = new int[n];
    Arrays.fill(ans, -1);
    for (int i = 0; i < n; i++) {
        for (int step = 1; step < n; step++) {
            int j = (i + step) % n;
            if (nums[j] > nums[i]) { ans[i] = nums[j]; break; }
        }
    }
    return ans;
}
```

Optimized (monotonic stack of indices, $2n$ pass)

Each index is pushed/popped once; wrap by iterating twice.

```

public int[] nextGreaterElements(int[] nums) {
    int n = nums.length;
    int[] ans = new int[n];
    Arrays.fill(ans, -1);
    Deque<Integer> stack = new ArrayDeque<>(); // holds indices, values decreasing
    for (int i = 0; i < 2 * n; i++) {
        int cur = nums[i % n];
        while (!stack.isEmpty() && nums[stack.peek()] < cur) {
            ans[stack.pop()] = cur; // cur is the next-greater for the popped
        }
        if (i < n) stack.push(i); // only push real (first-pass) indices
    }
    return ans;
}

```

Version	Time	Space
Brute force	$O(n^2)$	$O(1)$
Monotonic stack	$O(n)$	$O(n)$

3.2 Largest Rectangle in Histogram (LC 84)

Problem. Given an array `heights` representing the bar heights of a histogram (each bar has width 1, placed side by side with no gaps), find the area of the **largest rectangle** that fits entirely under the histogram's outline. (*Honestly Hard — the flagship monotonic-stack problem.*)

- **Constraints:** $1 \leq \text{heights.length} \leq 10^5$; $0 \leq \text{heights}[i] \leq 10^4$.
- **Clarifying assumptions:** bars can have height 0 (they contribute no area but don't break contiguity of taller neighbors on either side); the rectangle's width must be a contiguous run of bars, and its height is capped by the **shortest** bar in that run.
- **Why it's tricky:** the naive "expand from every bar" idea is $O(n^2)$; the $O(n)$ insight requires realizing that the right boundary for a bar is only known the *moment a shorter bar shows up later* — which is exactly when a monotonic stack pops it.

Example.

```

Input:  [2, 1, 5, 6, 2, 3]
Output: 10      (bars of height 5 and 6 span width 2 -> 5*2 = 10)

```

Intuition (diagram). A rectangle's height is limited by its **shortest** bar. For each bar, ask: *how far left and right can I extend before hitting a shorter bar?* A monotonic **increasing** stack (of indices, values increasing bottom to top) finds both boundaries: when a shorter (or equal) bar arrives, every taller bar sitting on top of the stack "closes out" — **the moment we pop it is the moment its right boundary becomes known** (the current index is the first bar to its right that is too short to extend the rectangle), and its left boundary is whatever index is now exposed underneath it on the stack (the nearest bar to the left that is shorter).

```

height 6 |           ##
height 5 |         ##  ##
height 4 |         ##  ##
height 3 |         ##  ##    ##
height 2 |    ##    ##  ##  ##  ##
height 1 |    ##  ##  ##  ##  ##  ##
          +-----+
index    |    0    1    2    3    4    5
height   |    2    1    5    6    2    3

```

When index 4 (height 2) arrives, bars at index 3 (height 6) and index 2 (height 5) get popped, because both are taller than 2 and can't extend any further right:

```

pop idx 3 (height 6): left boundary = idx 2 (now on top) -> width = 4 - 2 - 1 = 1, area =
pop idx 2 (height 5): left boundary = idx 1 (now on top) -> width = 4 - 1 - 1 = 2, area =

```

Approach (step by step)

1. **Invariant:** the stack holds indices with **strictly increasing heights** from bottom to top. This means: whatever is on top of the stack is the tallest bar seen so far that hasn't yet found its right boundary.
2. **The key insight — "popping = finding the right boundary":** we only pop index `j` when we hit some later index `i` with `heights[i] <= heights[j]`. That means `i` is the first bar to the right of `j` that is not taller than it — i.e., `i` is `j`'s right boundary. Symmetrically, once `j` is popped, whatever remains on top of the stack (call it `k`) is the nearest bar to `j`'s left that is shorter than `j` — i.e., `j`'s left boundary. So the width of the rectangle capped at height `heights[j]` is `i - k - 1`.
3. **Why each index is pushed and popped exactly once ($O(n)$ argument):** every index enters the stack via a single `push` and, once popped, is discarded forever (never re-pushed). So across the whole scan, total pushes = `n` and total pops $\leq n$; the `while` loop's work is bounded by the total number of pops, giving amortized $O(1)$ per index and $O(n)$ overall.
4. Append a sentinel height of `0` after the real array (`i == n`) so that the final pass forces every remaining bar off the stack and its area gets counted.
5. For each `i` from `0` to `n` (inclusive, using the sentinel at `i == n`): while the stack is non-empty and the height at the top is $\geq$ the current height, pop it, compute its width as `i - (new top, or -1 if empty) - 1`, and update `best`.
6. Push `i` onto the stack (its right boundary isn't known yet).

Worked trace on `heights = [2, 1, 5, 6, 2, 3]` (sentinel `0` appended at `i = 6`):

i	h	stack (before)	action
0	2	[]	push 0 → [0]
1	1	[0]	pop 0: width=1-(-1)-1=1, area=2*1=2 (best=2) → []; push 1 → [1]
2	5	[1]	5>1, stop → push 2 → [1,2]
3	6	[1,2]	6>5, stop → push 3 → [1,2,3]
4	2	[1,2,3]	pop 3: width=4-2-1=1, area=6*1=6 pop 2: width=4-1-1=2, area=5*2=10 (best=10) → [1]; push 4 → [1,4]
5	3	[1,4]	3>2, stop → push 5 → [1,4,5]
6	0	[1,4,5]	pop 5: width=6-4-1=1, area=3*1=3 pop 4: width=6-1-1=4, area=2*4=8 pop 1: width=6-(-1)-1=6, area=1*6=6 → []; push 6 → [6]

Best area found is **10** , matching the expected output.

Brute force

For each pair of bars, the rectangle height is the min in between.

```
public int largestRectangleAreaBrute(int[] heights) {
    int best = 0;
    for (int i = 0; i < heights.length; i++) {
        int minHeight = heights[i];
        for (int j = i; j < heights.length; j++) {
            minHeight = Math.min(minHeight, heights[j]);
            best = Math.max(best, minHeight * (j - i + 1));
        }
    }
    return best;
}
```

Optimized (monotonic increasing stack)

A sentinel **0** at the end flushes the stack. Widths come from index gaps.

```
public int largestRectangleArea(int[] heights) {
    Deque<Integer> stack = new ArrayDeque<>(); // indices with increasing heights
    int best = 0, n = heights.length;
    for (int i = 0; i <= n; i++) {
        int h = (i == n) ? 0 : heights[i]; // final 0 forces everything to pop
        while (!stack.isEmpty() && heights[stack.peek()] >= h) {
            int height = heights[stack.pop()];
            int leftBoundary = stack.isEmpty() ? -1 : stack.peek();
            int width = i - leftBoundary - 1; // between the new left bound and current
            best = Math.max(best, height * width);
        }
        stack.push(i);
    }
    return best;
}
```

Version	Time	Space
Brute force	$O(n^2)$	$O(1)$
Monotonic stack	$O(n)$	$O(n)$

3.3 Sliding Window Maximum (LC 239)

Problem. Given an integer array `nums` and a window size `k`, slide a window of length `k` from the left edge of the array to the right, one step at a time, and return the maximum value in the window at each position.

- **Constraints:** $1 \leq \text{nums.length} \leq 10^5$; $-10^4 \leq \text{nums}[i] \leq 10^4$; $1 \leq k \leq \text{nums.length}$.
- **Clarifying assumptions:** the window always has exactly `k` elements (it starts fully inside the array and only stops sliding when it would run off the end); duplicates are allowed, and the output has `nums.length - k + 1` entries.
- **Why it's tricky:** a naive re-scan of each window is $O(n \cdot k)$; a naive "recompute max from scratch" also can't cheaply handle the max *leaving* the window. The trick is a structure that discards values that can never be the max again, so it never needs to look at a stale value.

Example.

Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`
Output: `[3, 3, 5, 5, 6, 7]`

Intuition (diagram — monotonic deque). Keep a deque of **indices** whose values are in **decreasing** order from front to back. The front is always the current window's max. Two eviction rules keep this true: pop from the **back** any index whose value is $\leq$ the incoming value (a smaller-or-equal value that came earlier can never be the max once a bigger value is in the window, so it's useless — discard it), and pop from the **front** any index that has slid out the left edge of the window (it's stale, not because of its value, but because of its position).

Approach (step by step)

1. **Invariant:** the deque holds indices $i_0 < i_1 < \dots < i_k$ with $\text{nums}[i_0] > \text{nums}[i_1] > \dots > \text{nums}[i_k]$ — strictly decreasing values from front to back. The front (i_0) is always the index of the maximum value currently in the window.
2. **Why each index is pushed/popped once ($O(n)$ argument):** each index is added to the back exactly once. It can later be removed either from the back (a "dominated" pop, triggered by a bigger incoming value) or from the front (an "expired" pop, triggered by the window moving past it) — but either way, once removed it's gone for good. Total insertions are bounded by `n`, so total removals are too, making the whole scan $O(n)$ amortized.
3. For each index `i` from `0` to `n - 1`: a. **Evict from front** if the front index is $\leq i - k$ (it has fallen out of the window $[i - k + 1, i]$). b. **Evict from back** while the value at the back index is $\leq \text{nums}[i]$ (those values can never beat `nums[i]` while `i` is still in the window). c. Push `i`

onto the back. d. Once the window is fully formed ($i \geq k - 1$), the front of the deque is the max for the window ending at i .

Worked trace on `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`:

i	val	deque (before)	action	
0	1	[]	push 0	-> [0]
1	3	[0]	pop back 0 ($\text{nums}[0]=1 \leq 3$)	-> []; push 1 -> [1]
2	-1	[1]	$-1 < \text{nums}[1]=3$, no pop	-> push 2 -> [1,2] max=3
3	-3	[1,2]	$-3 < \text{nums}[2]=-1$, no pop	-> push 3 -> [1,2,3] max=3
4	5	[1,2,3]	evict front 1 ($1 \leq 4-3=1$)	-> [2,3]
			pop back 3,2 ($5 \geq \text{both}$)	-> []; push 4 -> [4] max=5
5	3	[4]	$3 < \text{nums}[4]=5$, no pop	-> push 5 -> [4,5] max=5
6	6	[4,5]	pop back 5,4 ($6 \geq \text{both}$)	-> []; push 6 -> [6] max=6
7	7	[6]	pop back 6 ($7 \geq 6$)	-> []; push 7 -> [7] max=7

Result: `[3, 3, 5, 5, 6, 7]`, matching the expected output.

Brute force

Scan each window for its max.

```
public int[] maxSlidingWindowBrute(int[] nums, int k) {
    int n = nums.length;
    int[] ans = new int[n - k + 1];
    for (int i = 0; i + k <= n; i++) {
        int mx = nums[i];
        for (int j = i; j < i + k; j++) mx = Math.max(mx, nums[j]);
        ans[i] = mx;
    }
    return ans;
}
```

Optimized (monotonic deque of indices)

Front is the max; maintain decreasing order and evict stale indices.

```
public int[] maxSlidingWindow(int[] nums, int k) {
    int n = nums.length;
    int[] ans = new int[n - k + 1];
    Deque<Integer> dq = new ArrayDeque<>(); // indices, values decreasing
    for (int i = 0; i < n; i++) {
        if (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst(); // evict out-of-window
        while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i]) dq.pollLast(); // keep decreasing
        dq.offerLast(i);
        if (i >= k - 1) ans[i - k + 1] = nums[dq.peekFirst()]; // front = window max
    }
    return ans;
}
```

Version	Time	Space
Brute force	$O(n \cdot k)$	$O(1)$
Monotonic deque	$O(n)$	$O(k)$

3.4 Sum of Subarray Minimums (LC 907)

Problem. Given an integer array `arr`, consider **every** contiguous subarray. For each one, take its minimum value. Return the sum of all those minimums, modulo `1_000_000_007` (the number of subarrays is $O(n^2)$, so the sum can be huge — that's why the modulo).

- **Constraints:** `1 <= arr.length <= 3 * 10^4`; `1 <= arr[i] <= 3 * 10^4`.
- **Clarifying assumptions:** subarrays are contiguous (not subsequences) and every subarray of every length is counted, including single elements; duplicate values are allowed and must not be double-counted.
- **Why it's tricky:** the natural approach enumerates $O(n^2)$ subarrays; the $O(n)$ trick requires flipping the question from "what is the min of this subarray" to "for how many subarrays is *this element* the min" — a **contribution-counting** technique — and then handling duplicate values carefully so no subarray's minimum is counted twice.

Example.

```
Input:  [3, 1, 2, 4]
Output: 17
```

Intuition (diagram — contribution counting). Instead of enumerating subarrays, ask for each element: *in how many subarrays am I the minimum?* That's (count of positions I can extend left) $\times$ (count of positions I can extend right). A monotonic stack finds, for each element, the distance to the **previous smaller** element and the **next smaller-or-equal** element; those distances multiply to its subarray count, and each element contributes `value x leftSpan x rightSpan`.

```
arr:    3    1    2    4
index:  0    1    2    3

for value 1 (index 1): nothing to its left is smaller (span left = 2: itself + edge),
                      nothing to its right is smaller (span right = 3: itself + edge)
  1 is the min of 2*3 = 6 subarrays -> contributes 1*6 = 6
(use "strictly smaller" on the left scan and "smaller-or-equal" on the right scan so
 that equal values are never both credited with the same subarray)
```

Approach (step by step)

1. **Key idea:** total answer = sum over every index `i` of `arr[i] * left[i] * right[i]`, where `left[i]` counts how many subarrays ending exactly at some point but *starting* somewhere in `(prevSmaller, i]` still have `arr[i]` as the minimum, and `right[i]` is the mirror for the right side.
2. **Invariant for the left pass:** scan left to right keeping a stack of indices with **strictly increasing** values (top = most recent, smallest-so-far-unresolved). For index `i`, pop everything with a value `> arr[i]` (they can't be a "previous smaller" for anything further right once `arr[i]` shows up); whatever remains on top is the nearest **strictly smaller** element to the left. `left[i] = i - (that index)`, or `i + 1` if the stack is empty (nothing smaller to the left at all).
3. **Invariant for the right pass:** scan right to left the same way, but pop on `>=` instead of `>` — this asymmetry (strict on one side, non-strict on the other) is what prevents double-counting when

duplicate values exist. $\text{right}[i] = (\text{that index}) - i$, or $n - i$ if the stack empties out.

4. **Why each index is pushed/popped once per pass ($O(n)$ argument):** exactly as with the other problems, each index enters a stack once and leaves at most once per direction, so each of the two passes is $O(n)$, and the whole algorithm is $O(n)$.
5. Sum $\text{arr}[i] * \text{left}[i] * \text{right}[i] \pmod{1e9+7}$ over all i .

Worked trace on $\text{arr} = [3, 1, 2, 4]$ (left pass uses "pop while $>$ ", right pass uses "pop while $\geq$ "):

i	arr[i]	left[i]	right[i]	contribution = arr[i]*left[i]*right[i]
0	3	1	1	3*1*1 = 3
1	1	2	3	1*2*3 = 6
2	2	1	2	2*1*2 = 4
3	4	1	1	4*1*1 = 4

Total = $3 + 6 + 4 + 4 = 17$, matching the expected output.

Brute force

Every subarray, track its running minimum.

```
public int sumSubarrayMinsBrute(int[] arr) {
    long MOD = 1_000_000_007L, total = 0;
    for (int i = 0; i < arr.length; i++) {
        int mn = arr[i];
        for (int j = i; j < arr.length; j++) {
            mn = Math.min(mn, arr[j]);
            total = (total + mn) % MOD;
        }
    }
    return (int) total;
}
```

Optimized (monotonic stack, contribution of each element)

Left span uses "strictly greater on the stack", right span "greater-or-equal", so equal minimums are counted exactly once.


```

public int sumSubarrayMins(int[] arr) {
    long MOD = 1_000_000_007L, total = 0;
    int n = arr.length;
    int[] left = new int[n];    // # of subarrays extending left where arr[i] stays the min
    int[] right = new int[n];   // # extending right
    Deque<Integer> stack = new ArrayDeque<>();

    for (int i = 0; i < n; i++) {
        // previous strictly-smaller
        while (!stack.isEmpty() && arr[stack.peek()] > arr[i]) stack.pop();
        left[i] = stack.isEmpty() ? i + 1 : i - stack.peek();
        stack.push(i);
    }
    stack.clear();
    for (int i = n - 1; i >= 0; i--) {
        // next smaller-or-equal
        while (!stack.isEmpty() && arr[stack.peek()] >= arr[i]) stack.pop();
        right[i] = stack.isEmpty() ? n - i : stack.peek() - i;
        stack.push(i);
    }
    for (int i = 0; i < n; i++) {
        total = (total + (long) arr[i] * left[i] % MOD * right[i]) % MOD;
    }
    return (int) total;
}

```

Version	Time	Space
Brute force	$O(n^2)$	$O(1)$
Monotonic stack	$O(n)$	$O(n)$

3.5 Online Stock Span (LC 901)

Problem. Design a data structure that models a stream of daily stock prices, one call at a time (`next(price)`), and for each new price returns its **span**: the number of consecutive days ending with today (i.e. going backwards from today) whose price was **less than or equal to** today's price.

- **Constraints:** up to 10^4 calls to `next`; $1 \leq \text{price} \leq 10^5$.
- **Clarifying assumptions:** "online" means prices arrive one at a time and you must answer immediately after each one — you can't look ahead; ties count ($\leq$, not $<$), so a run of equal prices all extend each other's span.
- **Why it's tricky:** the naive approach re-scans backwards through all history on every call, degrading to $O(n)$ per call ($O(n^2)$ overall). The insight is that once a day is "absorbed" by a later, taller price, it can never start a fresh maximal run again, so it can be discarded from future consideration — that's what makes a stack the right fit for a *streaming* (not batch) problem.

Example.

```

prices: 100 80 60 70 60 75 85
spans:  1  1  1  2  1  4  6

```

Intuition (diagram). When today's price is $\geq$ a recent day's price, that day is "absorbed" into today's span — it can never anchor its own longer run again once something taller/equal has come after it. Keep

a stack of (price, span) pairs, price decreasing from bottom to top. On each new price, pop every day whose price is $\leq$ today's, folding their spans into today's running total (their days are now "inside" today's streak) — then push (today's price, total span).

Approach (step by step)

1. **Invariant:** the stack holds (price, span) pairs with strictly **decreasing** prices from bottom to top, and each span already accounts for everything that pair previously absorbed. The top of the stack is the most recent day that hasn't yet been absorbed.
2. **Why each entry is pushed/popped at most once (O(1) amortized per call):** each call pushes exactly one new (price, span) pair. An entry is popped only when a later price absorbs it, and once popped it is gone forever (its span lives on folded into the new entry). So across all next calls, total pushes = total calls, and total pops $\leq$ total pushes — the same amortization argument as the other problems, just spread across separate method calls instead of one loop.
3. On next(price) : start span = 1 (today always covers at least itself).
4. While the stack is non-empty and the top's price is $\leq$ price : pop it and add its stored span into span (those days are absorbed — they extend today's run, not their own).
5. Push (price, span) onto the stack, and return span.

Worked trace on prices = [100, 80, 60, 70, 60, 75, 85] :

price	stack (before)	action	span
100	[]	push (100,1) → [(100,1)]	1
80	[(100,1)]	100≤80? no, stop; push (80,1) → [...,(80,1)]	1
60	[(100,1),(80,1)]	80≤60? no, stop; push (60,1) → [...,(60,1)]	1
70	[...,(80,1),(60,1)]	pop (60,1): span=1+1=2; 80≤70? no, stop push (70,2) → [...,(70,2)]	2
60	[...,(80,1),(70,2)]	70≤60? no, stop; push (60,1) → [...,(60,1)]	1
75	[...,(70,2),(60,1)]	pop (60,1): span=1+1=2 pop (70,2): span=2+2=4; 80≤75? no, stop push (75,4) → [...,(75,4)]	4
85	[...,(80,1),(75,4)]	pop (75,4): span=1+4=5 pop (80,1): span=5+1=6; 100≤85? no, stop push (85,6) → [...,(85,6)]	6

Spans returned in order: 1, 1, 1, 2, 1, 4, 6, matching the expected output.

Brute force

Store all prices; walk backwards each call.

```
class StockSpannerBrute {
    private final List<Integer> prices = new ArrayList<>();

    public int next(int price) {
        prices.add(price);
        int span = 0;
        for (int i = prices.size() - 1; i >= 0 && prices.get(i) <= price; i--) span++;
        return span;
    }
}
```

Optimized (monotonic stack of price+span)

Each price is pushed and popped at most once → amortized $O(1)$ per call.

```
class StockSpanner {
    private final Deque<int[]> stack = new ArrayDeque<>();    // {price, accumulatedSpan}

    public int next(int price) {
        int span = 1;
        while (!stack.isEmpty() && stack.peek()[0] <= price) {
            span += stack.pop()[1];        // absorb the span of every day this price dominates
        }
        stack.push(new int[] { price, span });
        return span;
    }
}
```

Version	Time	Space
Brute force	$O(n)$ per call	$O(n)$
Monotonic stack	$O(1)$ amortized per call	$O(n)$

4. Advanced Dynamic Programming

The core idea. DP is "solve once, remember, reuse." Vol 2 covered the linear and grid classics. This section adds the three DP *shapes* that trip people up in the second round: **interval DP** (the answer for a range depends on splitting it — think "burst / merge from the inside out"), **2D string DP** (align two sequences on a grid), and **state-machine DP** (each position carries a small state, like holding vs. not-holding a stock). The recipe is always: define the state in one English sentence, write the recurrence, pick an iteration order so dependencies are ready, then (optionally) shrink the memory.

Reach for it when: "count the ways / best value", overlapping subproblems, choices that compound, or a range/pair/state that can be described with 1–3 small indices.

4.1 Palindromic Substrings (LC 647)

Problem. Given a string `s`, return the number of **substrings** of `s` that are palindromes. A substring is contiguous; two substrings that occupy different positions count separately even if they contain the same characters (e.g. in `"aaa"` the two length-1 `"a"` substrings at index 0 and index 1 are counted separately).

- **Constraints:** `1 <= s.length <= 1000`; `s` consists of lowercase English letters only.
- **Clarifying assumptions:** overlapping and repeated palindromic substrings both count (this is a count of *occurrences*, not distinct palindromic strings); the empty string is not a substring to consider.
- **Why it's tricky:** the natural instinct is to check every substring for palindrome-ness ($O(n^3)$), but the real insight is that every palindrome is uniquely identified by its **center** — so you can generate palindromes instead of testing them, dropping to $O(n^2)$.

Example.

```
Input:  "aaa"
Output: 6      ("a","a","a","aa","aa","aaa")
```

Intuition (diagram — expand around center). Every palindrome has a center: either a single character (odd length) or a gap between two characters (even length). There are `2n-1` centers. Expand outward from each while the two ends match; every successful expansion is one more palindrome.

```
"aba"
center at 'b' (index 1): expand -> a...a match -> "aba"  (+1, plus "b" itself +1)
center at 'a' (index 0): just "a"                        (+1)
center at 'a' (index 2): just "a"                        (+1)
centers between letters (even): "ab" no, "ba" no
total = 4
```

Approach (step by step)

Key idea / invariant. Instead of asking "is this substring a palindrome?" for every substring, pick every possible **center** and grow outward. A center is either one character (odd-length palindrome) or the gap between two adjacent characters (even-length palindrome), so there are exactly $2n - 1$ centers for a string of length n . As long as the two sides of a center still match, the substring between them is a palindrome, and every further match one step out is *another, longer* palindrome — so each successful expansion adds exactly 1 to the count.

Numbered walk-through. 1. Loop `center` from `0` to `2n - 2`. 2. Decode the center into a `(lo, hi)` pair: `lo = center / 2`, `hi = lo + (center % 2)`. Even `center` $\rightarrow$ `lo == hi` (odd-length palindrome, single character start). Odd `center` $\rightarrow$ `hi == lo + 1` (even-length palindrome, gap between two characters). 3. While `lo >= 0`, `hi < n`, and `s[lo] == s[hi]`: this is a valid palindrome, so increment the count, then move outward (`lo--`, `hi++`) and check again. 4. Stop expanding as soon as the characters mismatch or a bound is exceeded; move to the next center. 5. Sum the counts from all $2n - 1$ centers.

Base case: every single character (`lo == hi`) is trivially a palindrome of length 1 — this is the first thing each odd center matches.

Worked trace on "aaa" ($n = 3$, indices 0,1,2):

center 0 (<code>lo=0,hi=0</code> odd):	"a"	$\rightarrow +1$	running=1
center 1 (<code>lo=0,hi=1</code> even):	"aa"	$\rightarrow +1$	running=2
center 2 (<code>lo=1,hi=1</code> odd):	"a" then "aaa"	$\rightarrow +1, +1$	running=4
center 3 (<code>lo=1,hi=2</code> even):	"aa"	$\rightarrow +1$	running=5
center 4 (<code>lo=2,hi=2</code> odd):	"a"	$\rightarrow +1$	running=6
total = 6			

Brute force

Check every substring for palindrome-ness.

```
public int countSubstringsBrute(String s) {
    int n = s.length(), count = 0;
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            if (isPalindrome(s, i, j)) count++;
        }
    }
    return count;
}

private boolean isPalindrome(String s, int lo, int hi) {
    while (lo < hi) {
        if (s.charAt(lo++) != s.charAt(hi--)) return false;
    }
    return true;
}
```

Optimized (expand around each center)

Two centers per index (odd and even); each expansion is $O(1)$ work.

```

public int countSubstrings(String s) {
    int n = s.length(), count = 0;
    for (int center = 0; center < 2 * n - 1; center++) {
        int lo = center / 2;
        int hi = lo + (center % 2); // even center straddles two chars
        while (lo >= 0 && hi < n && s.charAt(lo) == s.charAt(hi)) {
            count++; // each valid expansion is one palindrome
            lo--;
            hi++;
        }
    }
    return count;
}

```

Version	Time	Space
Brute force	$O(n^3)$	$O(1)$
Expand around center	$O(n^2)$	$O(1)$

4.2 Best Time to Buy and Sell Stock with Cooldown (LC 309)

Problem. Given an array `prices` where `prices[i]` is the stock's price on day `i`, return the maximum profit achievable if you may complete as many buy/sell transactions as you like, subject to: (1) you must sell before you buy again (you never hold more than one share), and (2) after you sell, you must **cooldown for one full day** before you may buy again.

- **Constraints:** `1 <= prices.length <= 5000`; `0 <= prices[i] <= 1000`.
- **Clarifying assumptions:** you may choose to make zero transactions (answer is then `0`); you cannot buy and sell on the same day; the cooldown is exactly one day (you can buy again on the second day after a sale).
- **Why it's tricky:** it looks like a greedy "buy every dip, sell every peak" problem, but the cooldown constraint couples consecutive decisions together — you need a **state machine** (hold / sold / rest) rather than a single running total, or you'll either forget the cooldown or double-count a transaction.

Example.

```

Input: prices = [1, 2, 3, 0, 2]
Output: 3      (buy1, sell2 -> cooldown -> buy0, sell2)

```

Intuition (diagram — state machine). Each day you're in exactly one of three states: **hold** (own a share), **sold** (just sold today — must rest tomorrow), or **rest** (idle, free to buy). Draw the transitions and the DP writes itself.

```

      buy (-price)
rest -----> hold
  ^ \      |
  |  \ stay | sell (+price)
  |   v     v
rest <----- sold      (sold -> rest is the forced cooldown)
      (do nothing)

hold[i] = max(hold[i-1], rest[i-1] - price) // keep holding, or buy today
sold[i] = hold[i-1] + price                // sell today
rest[i] = max(rest[i-1], sold[i-1])        // stay idle, or come off cooldown

```

Approach (step by step)

Key idea / invariant. On every day, track the best possible profit **so far** for each of the three states a day can end in: - **hold** = max profit if today you are holding a share (either bought before, or bought today). - **sold** = max profit if today you just sold your share (so tomorrow you are forced to rest). - **rest** = max profit if today you hold nothing and are free to buy (either you've been idle, or your cooldown from a sale just ended).

The invariant is that **hold**, **sold**, **rest** always hold the best achievable profit for *that state as of the current day*, computed only from *yesterday's* three values — this is what lets the DP run in $O(1)$ space.

Recurrence in words, and what each state means: - **hold[i]** = $\max(\text{hold}[i-1], \text{rest}[i-1] - \text{price}[i])$ — either keep holding what you already held, or buy today starting from yesterday's rested state (pay $\text{price}[i]$). - **sold[i]** = $\text{hold}[i-1] + \text{price}[i]$ — the only way to be "just sold" today is to have held yesterday and sell today. - **rest[i]** = $\max(\text{rest}[i-1], \text{sold}[i-1])$ — either you were already resting, or yesterday you sold and today is your forced cooldown day (which looks identical to "resting").

Base case (day 0, before any prices are processed): **hold** = $-\infty$ (impossible to be holding before buying anything), **sold** = 0, **rest** = 0.

Fill order: left to right over **prices**, one day at a time, each day depending only on the previous day's three values (roll them forward, don't keep a full table).

Final answer: $\max(\text{sold}, \text{rest})$ at the end — you should never end the array still **hold**ing, since selling at the end (or not buying at all) can never be worse.

Worked trace on `prices = [1, 2, 3, 0, 2]`:

day	price	hold	sold	rest
(init)	-	$-\infty$	0	0
0	1	-1	$-\infty$	0
1	2	-1	1	0
2	3	-1	2	1
3	0	1	-1	2
4	2	1	3	2

answer = $\max(\text{sold}, \text{rest}) = \max(3, 2) = 3$

(the winning line: buy day 0 (price 1), sell day 1 (price 2) for +1, forced cooldown on day 2, buy day 3 (price 0), sell day 4 (price 2) for +2 -> total profit 3, matching the example)

Brute force

Recurse over "buy or skip" at every day (exponential).

```
public int maxProfitBrute(int[] prices) {
    return dfs(prices, 0, false);    // false = not currently holding
}

private int dfs(int[] prices, int day, boolean holding) {
    if (day >= prices.length) return 0;
    int skip = dfs(prices, day + 1, holding);
    if (holding) {
        int sell = prices[day] + dfs(prices, day + 2, false);    // +2 enforces cooldown
        return Math.max(skip, sell);
    } else {
        int buy = -prices[day] + dfs(prices, day + 1, true);
        return Math.max(skip, buy);
    }
}
```

Optimized (three rolling states)

Carry just yesterday's three numbers.

```
public int maxProfit(int[] prices) {
    int hold = Integer.MIN_VALUE;    // best profit while holding a share
    int sold = 0;                    // best profit having just sold today
    int rest = 0;                    // best profit while idle (free to buy)
    for (int price : prices) {
        int prevSold = sold;
        sold = hold + price;          // sell what we held
        hold = Math.max(hold, rest - price);    // keep holding, or buy from rest
        rest = Math.max(rest, prevSold);    // stay idle, or finish cooldown
    }
    return Math.max(sold, rest);      // never end still holding
}
```

Version	Time	Space
Brute recursion	$O(2^n)$	$O(n)$ stack
State machine	$O(n)$	$O(1)$

4.3 Best Time to Buy and Sell Stock IV (LC 188)

Problem. Given an integer `k` and an array `prices`, return the maximum profit achievable with **at most `k` transactions**, where a transaction is one buy followed by one later sell and you must sell your current share before buying another. (*Hard — the general case of the stock family; §4.2's cooldown problem is a restricted variant, and "unlimited transactions" is the special case $k \geq n/2$.*)

- **Constraints:** $1 \leq k \leq 100$; $1 \leq \text{prices.length} \leq 1000$; $0 \leq \text{prices}[i] \leq 1000$.
- **Clarifying assumptions:** "at most `k`" means you may use fewer than `k` transactions if that's better (e.g. a strictly falling price series should return `0`); you cannot hold more than one share at

a time; buying and selling on the same day counts as no transaction (or, equivalently, is never beneficial and can be ignored).

- **Why it's tricky:** the state space needs a *transaction counter* in addition to the usual day index, so the naive recursion is $O(n * k)$ branches deep with no obvious rolling trick at first glance — and if k is large relative to n , the "at most k " DP degenerates into needless work, so the optimized solution needs a size check to fall back to the simpler unlimited-transactions greedy.

Example.

```
Input: k = 2, prices = [3, 2, 6, 5, 0, 3]
Output: 7          (buy2 sell6 = 4, buy0 sell3 = 3)
```

Intuition (diagram). Track, for each transaction count t from $1..k$, two values: the best profit after **buying** the t -th share and after **selling** it. Each day updates them. $buy[t]$ wants to minimize cost given profit already banked from $t-1$ sells; $sell[t]$ cashes out. If k is huge ($\geq n/2$) it's unlimited transactions — sum every upward step.

```
dp per transaction t:
    buy[t] = max(buy[t], sell[t-1] - price) // spend, but keep profit from earlier trade
    sell[t] = max(sell[t], buy[t] + price)  // realize this trade's gain

table columns = transactions 1..k, updated day by day
```

Approach (step by step)

Key idea / invariant. For each transaction slot t ($1..k$), maintain two running best values as we scan the days left to right: - $buy[t]$ = best profit achievable so far having *bought* the t -th share (profit is negative or small — it's "money spent," offset by whatever was already banked from the first $t-1$ completed transactions). - $sell[t]$ = best profit achievable so far having *completed* the t -th transaction (bought and sold it).

The invariant: at every day, $buy[t]$ and $sell[t]$ are the best possible values *using only days seen so far*, and $buy[t]$ always builds on $sell[t-1]$ (you can't start transaction t with money from transaction t itself — only from the ones before it).

Recurrence in words: on each day, for every t from 1 to k , in increasing order (so that $buy[t]$ can see today's freshly-updated $sell[t-1]$ — transaction t may legitimately start on the very same day transaction $t-1$ finished): - $buy[t] = \max(buy[t], sell[t-1] - price)$ — either keep the best "just bought $\#t$ " position we already had, or start transaction t today by spending $price$, funded by whatever transaction $t-1$ had already banked. - $sell[t] = \max(sell[t], buy[t] + price)$ — either keep the best "just completed $\#t$ " position we already had, or cash out today's holding.

Base case: before any day is processed, $buy[t] = -\text{infinity}$ for all t (can't have bought yet) and $sell[t] = 0$ for all t (zero transactions completed costs/earns nothing); $sell[0] = 0$ always (a "0th transaction" trivially contributes no cost, which is what lets $buy[1]$ bootstrap off $sell[0]$).

Fill order: outer loop over days (left to right), inner loop over t from 1 to k (increasing, so $sell[t-1]$ reflects transaction $t-1$'s progress up through today).

The unlimited-transaction shortcut: if $k \geq n/2$, you can never actually use more than $n/2$ transactions anyway (each needs at least 2 distinct days), so the k cap is not binding — the answer is

just "sum every price increase," the same greedy as the unlimited version of this problem.

Worked trace on $k = 2$, $\text{prices} = [3, 2, 6, 5, 0, 3]$ (columns are `buy[1]`, `sell[1]`, `buy[2]`, `sell[2]` after processing each price; `-inf` abbreviated `-I`):

```
init          buy=[-I, -I]  sell=[0, 0]
price=3   buy[1]=-3   buy=[-3, -I]  sell=[0, 0]
price=2   buy[1]=-2   buy=[-2, -I]  sell=[0, 0]
price=6   sell[1]=4   buy=[-2, -2]  sell=[4, 4]
price=5   buy[2]=-1   buy=[-2, -1]  sell=[4, 4]
price=0   buy[2]= 4   buy=[-2,  4]  sell=[4, 4]
price=3   sell[2]=7   buy=[-2,  4]  sell=[4, 7]
answer = sell[2] = 7
```

(matches the example: buy at price 2 $\rightarrow$ sell at price 6 (+4), buy at price 0 $\rightarrow$ sell at price 3 (+3), total 7)

Brute force

Recurse over (day, transactions left, holding) — exponential without memoization.

```
public int maxProfitBrute(int k, int[] prices) {
    return dfs(prices, 0, k, false);
}

private int dfs(int[] prices, int day, int left, boolean holding) {
    if (day == prices.length || left == 0) return 0;
    int skip = dfs(prices, day + 1, left, holding);
    if (holding) {
        int sell = prices[day] + dfs(prices, day + 1, left - 1, false); // finishing a transaction
        return Math.max(skip, sell);
    } else {
        int buy = -prices[day] + dfs(prices, day + 1, left, true);
        return Math.max(skip, buy);
    }
}
```

Optimized (DP over k transactions, with the unlimited shortcut)

Two arrays of size `k+1`; special-case big `k`.

```

public int maxProfit(int k, int[] prices) {
    int n = prices.length;
    if (n == 0 || k == 0) return 0;
    if (k >= n / 2) { // effectively unlimited -> grab every rise
        int profit = 0;
        for (int i = 1; i < n; i++)
            if (prices[i] > prices[i - 1]) profit += prices[i] - prices[i - 1];
        return profit;
    }
    int[] buy = new int[k + 1];
    int[] sell = new int[k + 1];
    Arrays.fill(buy, Integer.MIN_VALUE); // haven't bought yet
    for (int price : prices) {
        for (int t = 1; t <= k; t++) {
            buy[t] = Math.max(buy[t], sell[t - 1] - price); // buy t-th share
            sell[t] = Math.max(sell[t], buy[t] + price); // sell it
        }
    }
    return sell[k];
}

```

Version	Time	Space
Brute recursion	$O(2^n)$	$O(n)$
DP over k	$O(n \cdot k)$	$O(k)$

4.4 Longest Common Substring (contiguous)

Problem. Given two strings `a` and `b`, return the length of the longest **substring** (a contiguous run of characters) that appears in both. (*Distinct from Vol 2's Longest Common Subsequence — here the shared characters must be adjacent in both strings, not just in the same relative order.*)

- **Constraints:** $1 \leq a.length, b.length \leq 1000$; both strings consist of lowercase English letters (or ASCII, depending on the exact source problem).
- **Clarifying assumptions:** if there is no common character at all, return `0`; ties (two common substrings of equal max length) only require the length, not which one; a single shared character counts as a common substring of length 1.
- **Why it's tricky:** it's easy to confuse this with Longest Common **Subsequence** and reuse its recurrence — but LCS's $dp[i][j] = dp[i-1][j-1] + 1$ on a match is *combined* with "or skip a character from either string" on a mismatch, which is wrong here: a mismatch must reset the run to `0`, not carry a max forward, because the substring must stay contiguous. The other trap: the answer is the **max value anywhere in the table**, not $dp[m][n]$.

Example.

```

Input: "abcde", "abfce"
Output: 2      ("ab", the only length-2 contiguous match; "ce" is not contiguous in "abfce")

```

Intuition (diagram — DP table). Let $dp[i][j]$ = length of the longest common substring that **ends exactly at** `a[i-1]` and `b[j-1]`. If those characters match, extend the diagonal: $dp[i][j] = dp[i-1][j-1] + 1$.

$[j-1] + 1$. If they differ, the run is broken $\rightarrow 0$. The answer is the largest cell anywhere in the table (not the corner, unlike LCS).

""	""	a	b	f	c	e	
""	0	0	0	0	0	0	
a	0	1	0	0	0	0	
b	0	0	2	0	0	0	$\leftarrow$ max cell: "ab" length 2
c	0	0	0	0	1	0	
d	0	0	0	0	0	0	
e	0	0	0	0	0	1	

match $\rightarrow dp[i][j] = dp[i-1][j-1] + 1$
 mismatch $\rightarrow dp[i][j] = 0$ (run is broken, does NOT carry forward like LCS)

Approach (step by step)

Key idea / invariant. $dp[i][j]$ means: "the length of the longest common substring that ends *exactly* at $a[i-1]$ and $b[j-1]$ " (1-indexed into the table, 0-indexed into the strings). This is a stricter invariant than LCS's "longest common subsequence using the first i and j characters" — here, a mismatch cannot be patched around by skipping a character, because the match must be contiguous, so it must drop straight to 0.

Recurrence in words: - If $a[i-1] == b[j-1]$: the characters match, so any common substring ending at $a[i-2]$, $b[j-2]$ can be extended by one character — $dp[i][j] = dp[i-1][j-1] + 1$. - If $a[i-1] != b[j-1]$: no common substring can end here at all, since the last characters disagree — $dp[i][j] = 0$.

Base case: row 0 and column 0 (representing an empty prefix of either string) are all 0, since there's nothing to match against an empty string.

Fill order: row by row (or column by column), i from 1 to m , j from 1 to n — each cell only needs $dp[i-1][j-1]$, i.e. the cell diagonally above-left, which is already computed.

Answer extraction: track a running $best = \max(best, dp[i][j])$ while filling — the overall answer is the largest value that ever appears anywhere in the table, not $dp[m][n]$ (the true longest common substring can end in the middle of both strings).

Worked trace on $a = "abcde"$, $b = "abfce"$: walking the table above, the diagonal run $dp["a"]["a"] = 1$ then $dp["b"]["b"] = dp["a"]["a"] + 1 = 2$ gives the substring "ab". Every other match ('c' at $a[2] / b[3]$, 'e' at $a[4] / b[4]$) is isolated (its diagonal neighbor is a mismatch), so it only reaches 1. The running max across the whole table is 2, which is the answer.

Brute force

Try every start pair, extend while characters match.

```

public int longestCommonSubstringBrute(String a, String b) {
    int best = 0;
    for (int i = 0; i < a.length(); i++) {
        for (int j = 0; j < b.length(); j++) {
            int len = 0;
            while (i + len < a.length() && j + len < b.length()
                && a.charAt(i + len) == b.charAt(j + len)) {
                len++;
            }
            best = Math.max(best, len);
        }
    }
    return best;
}

```

Optimized (2D DP on matching diagonals)

Fill the grid; track the running maximum.

```

public int longestCommonSubstring(String a, String b) {
    int m = a.length(), n = b.length(), best = 0;
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (a.charAt(i - 1) == b.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1] + 1; // extend the matching run diagonally
                best = Math.max(best, dp[i][j]);
            }
            // mismatch leaves dp[i][j] = 0: the run is broken
        }
    }
    return best;
}

```

Version	Time	Space
Brute force	$O(m \cdot n \cdot \min(m, n))$	$O(1)$
2D DP	$O(m \cdot n)$	$O(m \cdot n)$, reducible to $O(n)$

4.5 Minimum Insertion Steps to Make a String Palindrome (LC 1312)

Problem. Given a string `s`, return the minimum number of characters you must **insert** (anywhere, in any order) so that the resulting string is a palindrome.

- **Constraints:** $1 \leq s.length \leq 500$; `s` consists of lowercase English letters.
- **Clarifying assumptions:** insertions may be placed at any position, not just at the ends; a string that is already a palindrome needs `0` insertions; you never need to delete or change characters, only add.
- **Why it's tricky:** this is **interval DP** — the subproblem is defined on a *range* `s[i..j]`, not a prefix, so the recurrence looks inward from both ends simultaneously, and the fill order must go by

increasing interval length (you can't process `s[i..j]` until every strictly shorter interval inside it, like `s[i+1..j-1]`, is already solved).

Example.

```
Input: "leetcode"
Output: 5
```

Intuition (diagram — interval DP). `dp[i][j]` = insertions needed to fix the substring `s[i..j]`. If the ends match, they're already good $\rightarrow$ recurse inward: `dp[i+1][j-1]`. If they differ, insert a partner for one of them and take the cheaper side, plus one: `1 + min(dp[i+1][j], dp[i][j-1])`. Fill by increasing interval length so inner ranges are ready first.

```
"mbadm"      (dp[i][j] over widening intervals)
ends 'm'=='m'  $\rightarrow$  dp[0][4] = dp[1][3] (inner "bad")
"bad": b != d  $\rightarrow$  dp[1][3] = 1 + min(dp[2][3], dp[1][2]) = 1 + min(1, 1) = 2
answer = dp[0][4] = 2    (e.g. insert 'd' and 'b'  $\rightarrow$  "mdbadbm")
```

Approach (step by step)

Key idea / invariant. `dp[i][j]` = the minimum insertions needed to turn `s[i..j]` (inclusive, both ends included) into a palindrome, considered as a *standalone* string. The invariant is that `dp[i][j]` is fully determined by strictly smaller intervals nested inside it, so once every interval of length `< len` is solved, every interval of length `len` can be computed in $O(1)$.

Recurrence in words: - If `s[i] == s[j]`: the two ends already mirror each other, so they contribute nothing new — the answer for `s[i..j]` is exactly the answer for the inner interval `s[i+1..j-1]`: `dp[i][j] = dp[i+1][j-1]`. - If `s[i] != s[j]`: one of the two ends needs a matching partner inserted. You can either fix the left end by conceptually pairing it with a new character mirroring `s[j]` (shrinking the problem to `s[i][j-1]` ... i.e. `dp[i][j-1]`) or fix the right end symmetrically (`dp[i+1][j]`) — take whichever is cheaper, plus the one insertion just spent: `dp[i][j] = 1 + min(dp[i+1][j], dp[i][j-1])`.

Base case: any interval of length 1 (`i == j`, a single character) is already a palindrome: `dp[i][i] = 0`. (Length-0 "intervals" like `dp[i+1][j-1]` when `j = i+1` are also implicitly 0 — an empty range needs no insertions.)

Fill order: by **increasing interval length** `len` from 2 up to `n` (length-1 intervals are the base case). For each `len`, slide `i` from 0 to `n - len`, set `j = i + len - 1`. This guarantees `dp[i+1][j-1]`, `dp[i+1][j]`, and `dp[i][j-1]` — all intervals strictly shorter than `len` — are already filled before `dp[i][j]` is computed.

Worked trace on `s = "mbadm"` (`n = 5`, indices 0..4):

length 1 (base case): $dp[i][i] = 0$ for every i

length 2: $dp[0][1] = "mb" \rightarrow 1 + \min(dp[1][1], dp[0][0]) = 1$ $dp[1][2] = "ba" \rightarrow 1$ $dp[2][3] = "ad" \rightarrow 1$ $dp[3][4] = "cm" \rightarrow 1$

length 3: $dp[0][2] = "mba" \rightarrow m \neq a \rightarrow 1 + \min(dp[1][2], dp[0][1]) = 1 + \min(1, 1) = 2$
 $dp[1][3] = "bad" \rightarrow b \neq d \rightarrow 1 + \min(dp[2][3], dp[1][2]) = 1 + \min(1, 1) = 2$
 $dp[2][4] = "adm" \rightarrow a \neq m \rightarrow 1 + \min(dp[3][4], dp[2][3]) = 1 + \min(1, 1) = 2$

length 4: $dp[0][3] = "mbad" \rightarrow m \neq d \rightarrow 1 + \min(dp[1][3], dp[0][2]) = 1 + \min(2, 2) = 3$
 $dp[1][4] = "badm" \rightarrow b \neq m \rightarrow 1 + \min(dp[2][4], dp[1][3]) = 1 + \min(2, 2) = 3$

length 5: $dp[0][4] = "mbadm" \rightarrow m == m \rightarrow dp[1][3] = 2$

answer = $dp[0][4] = 2$

Filled table (. = below the diagonal, not computed — only $i \leq j$ is meaningful):

j:	0(m)	1(b)	2(a)	3(d)	4(m)
i=0(m)	0	1	2	3	2
i=1(b)	.	0	1	2	3
i=2(a)	.	.	0	1	2
i=3(d)	.	.	.	0	1
i=4(m)	.	.	.	.	0

Brute force

Recurse over the two ends without memoizing (exponential).

```
public int minInsertionsBrute(String s) {
    return dfs(s, 0, s.length() - 1);
}

private int dfs(String s, int i, int j) {
    if (i >= j) return 0;
    if (s.charAt(i) == s.charAt(j)) return dfs(s, i + 1, j - 1);
    return 1 + Math.min(dfs(s, i + 1, j), dfs(s, i, j - 1));
}
```

Optimized (interval DP, bottom-up)

Iterate over interval length so $dp[i+1][j-1]$ etc. are already filled.

```

public int minInsertions(String s) {
    int n = s.length();
    int[][] dp = new int[n][n]; // dp[i][j] = insertions to fix s[i..j]
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            if (s.charAt(i) == s.charAt(j)) {
                dp[i][j] = dp[i + 1][j - 1]; // ends already match
            } else {
                dp[i][j] = 1 + Math.min(dp[i + 1][j], dp[i][j - 1]); // pair off one end
            }
        }
    }
    return dp[0][n - 1];
}

```

Version	Time	Space
Brute recursion	$O(2^n)$	$O(n)$
Interval DP	$O(n^2)$	$O(n^2)$

4.6 Maximal Rectangle (LC 85)

Problem. Given a 2D binary matrix `matrix` filled with `'0'` and `'1'` characters, find the area of the largest rectangle (axis-aligned, made of contiguous rows and columns) containing only `1` s. (*Hard* — a beautiful fusion of DP and the "largest rectangle in histogram" monotonic-stack technique from §3.2.)

- **Constraints:** $1 \leq \text{rows}, \text{cols} \leq 200$; `matrix[i][j]` is either the character `'0'` or `'1'`.
- **Clarifying assumptions:** the rectangle must be a solid block of `1` s (no `0` s inside it); a single cell containing `1` is a valid rectangle of area 1; if the matrix is all `0` s, the answer is `0`.
- **Why it's tricky:** it looks like a 2D problem needing $O(\text{rows}^2 \cdot \text{cols}^2)$ brute enumeration of rectangle corners, but the key trick is a **dimensionality reduction**: treat each row as the "ground level" of a histogram (bar heights = how many consecutive `1` s stack upward into that row), and reuse the $O(\text{cols})$ monotonic-stack "largest rectangle in histogram" solution once per row — turning a 2D problem into `rows` many 1D problems.

Example.

```

Input:
1 0 1 0 0
1 0 1 1 1
1 1 1 1 1
1 0 0 1 0
Output: 6

```

Intuition (diagram). Process the matrix **row by row**. For each row, build a histogram where each column's bar height = the number of consecutive `1` s ending at that row (i.e. how far a solid column of `1` s stretches upward, including the current row). The answer for the whole matrix is the max "largest rectangle in histogram" over all rows — so this problem reduces to running §3.2 once per row.

row-by-row histograms (heights = consecutive 1s stacking upward through this row):

col:	0	1	2	3	4	
row0:	1	0	1	0	0	largestRectangleArea = 1
row1:	2	0	2	1	1	largestRectangleArea = 3
row2:	3	1	3	2	2	largestRectangleArea = 6 <- overall max
row3:	4	0	0	3	0	largestRectangleArea = 4

each row -> largestRectangleArea(heights); take the max over all rows -> 6

Approach (step by step)

Key idea / invariant. Reduce "largest all-1s rectangle in a 2D grid" to "largest rectangle in a histogram" (already solved in §3.2), applied once per row. The invariant maintained while scanning rows top to bottom: `heights[c]` always equals the number of consecutive 1s ending at the **current** row in column `c` (a run that resets to 0 the moment a 0 is seen). Because the top edge of any all-1s rectangle in the matrix must sit on some row's histogram, checking every row's histogram is guaranteed to find the true maximum.

Numbered walk-through. 1. Initialize `heights[0..cols-1] = 0`. 2. For each row of the matrix, top to bottom: a. Update `heights[c]` for every column `c`: if `matrix[row][c] == '1'`, increment `heights[c]` (the run of 1s grows by one more row); otherwise reset `heights[c] = 0` (the run is broken). b. Run the §3.2 monotonic-stack routine on the current `heights` array to get the largest rectangle whose *bottom edge* is this row — this covers every rectangle whose bottom row is exactly this row, regardless of how many rows tall it is. c. Update the running `best` with this row's result. 3. After all rows, `best` is the answer.

Why the per-row histogram trick is valid (the "state" it encodes): `heights[c]` is exactly the DP recurrence `heights[c] = matrix[row][c]=='1' ? heights_prev_row[c] + 1 : 0` — a 1D DP carried down the rows. It compresses "how tall a solid column of 1s is here" into one number, which is exactly the information the histogram algorithm needs; you don't need to remember *which* rows were 1s, only the count.

Worked trace on the example matrix, showing `heights` after each row and that row's largest-rectangle result (matches the diagram above): the maximum over all four rows is 6, achieved at `row2` with `heights = [3, 1, 3, 2, 2]` — the histogram algorithm finds bars 2 and 3 (0-indexed columns) each of height 2, spanning width 3 (columns 2, 3, 4), giving area `2 * 3 = 6`.

Brute force

For every cell treat it as a top-left corner and expand — very slow.

```

public int maximalRectangleBrute(char[][] matrix) {
    if (matrix.length == 0) return 0;
    int rows = matrix.length, cols = matrix[0].length, best = 0;
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (matrix[r][c] == '0') continue;
            int minWidth = Integer.MAX_VALUE;
            for (int down = r; down < rows; down++) {
                int width = 0;
                while (c + width < cols && matrix[down][c + width] == '1') width++;
                if (width == 0) break;
                minWidth = Math.min(minWidth, width);
                best = Math.max(best, minWidth * (down - r + 1));
            }
        }
    }
    return best;
}

```

Optimized (per-row histogram + monotonic stack)

Maintain column heights; run the histogram routine each row.

```

public int maximalRectangle(char[][] matrix) {
    if (matrix.length == 0) return 0;
    int cols = matrix[0].length, best = 0;
    int[] heights = new int[cols];
    for (char[] row : matrix) {
        for (int c = 0; c < cols; c++) {
            heights[c] = (row[c] == '1') ? heights[c] + 1 : 0; // grow or reset the bar
        }
        best = Math.max(best, largestRectangleArea(heights)); // reuse §3.2
    }
    return best;
}

private int largestRectangleArea(int[] heights) {
    Deque<Integer> stack = new ArrayDeque<>();
    int best = 0, n = heights.length;
    for (int i = 0; i <= n; i++) {
        int h = (i == n) ? 0 : heights[i];
        while (!stack.isEmpty() && heights[stack.peek()] >= h) {
            int height = heights[stack.pop()];
            int left = stack.isEmpty() ? -1 : stack.peek();
            best = Math.max(best, height * (i - left - 1));
        }
        stack.push(i);
    }
    return best;
}

```

Version	Time	Space
Brute force	$O((\text{rows} \cdot \text{cols})^2)$	$O(1)$
Row histogram + stack	$O(\text{rows} \cdot \text{cols})$	$O(\text{cols})$

5. Advanced Graphs

The core idea. Vol 1 taught grid BFS/DFS and Vol 3 added Dijkstra, union-find and topo sort. This section pushes into the graph techniques that a strong mid-level candidate is expected to reach for *without hesitation*: **Union-Find** as a reusable tool, **cycle detection** with three-color DFS, **multi-source BFS** and **0-1 BFS** (a deque trick that beats Dijkstra when edges are only 0 or 1), and **memoized DFS on a DAG** (DP over a graph).

Because you asked for it, every problem in this section leads with a picture — the graph or grid drawn out, the traversal order, or the union-find forest — *before* any Java. Sketch the same picture in the interview: it's the fastest way to convince yourself (and the interviewer) the approach is right.

Reach for it when: connectivity/components (union-find), "is there a cycle" / valid ordering (DFS colors, Kahn), shortest steps from *many* starts at once (multi-source BFS), shortest path where edges cost 0 or 1 (0-1 BFS), or "longest/most paths in a DAG" (memoized DFS).

5.1 Accounts Merge (LC 721)

Problem. You are given a list `accounts` where `accounts[i] = [name, email1, email2, ...]` is one account belonging to `name`. Two accounts belong to the **same person** if they share at least one email — and this relationship is **transitive** (if account A shares an email with B, and B shares a different email with C, then A, B, and C are all the same person, even though A and C may share no email directly). Merge all accounts belonging to the same person and return each merged account as a list: the name first, followed by all of that person's emails **sorted lexicographically**, with no duplicate emails.

- **Constraints (typical LeetCode bounds):** `1 <= accounts.length <= 1000`; `2 <= accounts[i].length <= 10`; email strings have length `<= 30` and contain exactly one '@'. The same email never belongs to two different people in the input.
- **Clarifying assumptions:** different people can share the same `name` (names are not unique — don't use the name to decide who owns which emails, only shared emails decide that); within the returned groups, the order of the groups themselves doesn't matter, only the sorted order of emails inside each group.
- **Why it's tricky:** the "same person" relationship can chain across *more than two* accounts through emails that never co-occur in a single account (A-B share one email, B-C share a different one) — a single pairwise merge pass misses that chain, which is exactly what union-find with path compression is built to handle in one linear pass.

Example.

```

Input:
["John", "a@x.com", "b@x.com"]
["John", "b@x.com", "c@x.com"]
["Mary", "m@x.com"]
Output:
["John", "a@x.com", "b@x.com", "c@x.com"]
["Mary", "m@x.com"]

```

Intuition (diagram — union-find over emails). Treat every **email** as a node, not the account. Within one account, union all its emails together. Shared emails then transitively connect accounts into one group. Finally, gather emails by their group's root.

```

account1: a -- b      union(a, b)
account2: b -- c      union(b, c) -> a, b, c now share one root
account3: m           stands alone (no shared email)

```

union-find forest after all merges:

```

      c      (root of a, b, c)           m      (root of itself)
     / \
    a   b

```

```

group {a, b, c} -> owner "John"      group {m} -> owner "Mary"

```

Approach (step by step)

Key idea / invariant. Every email is a node in a union-find (disjoint-set) structure. The invariant union-find maintains is: *two emails are in the same set if and only if they are known — directly or transitively — to belong to the same person*. We never need to track "which account" an email came from once it's merged; we only need each set's **root** and one name to label that root.

1. For every account, take its first email as an "anchor" and **union** every other email in that account with the anchor. This links all emails within one account into one set.
2. While doing so, record `owner[email] = name` for every email (any account touching that email will do — they all agree on the person once merged).
3. After processing all accounts, do one pass over every email and call `find(email)` to get its group's root; bucket emails into a **TreeSet** (sorted, deduped) keyed by that root.
4. For each root, prepend `owner[root]` to its sorted email list — that's one merged account.

`find` uses **path compression**: every time we walk up from a node to its root, we re-point that node directly at the root before returning. This flattens the tree so future `find` calls on the same node (or its descendants) are $O(1)$ -ish instead of re-walking the whole chain — across all calls this keeps the total cost near-linear (formally $O(E * \alpha(E))$, α being the inverse Ackermann function, effectively a small constant).

Worked trace on the example: - Account "John" #1: emails `a, b`. `union(a, b)` -> root becomes `b` (or `a`, order-dependent). - Account "John" #2: emails `b, c`. `union(b, c)` -> `b`'s root now points to `c`'s root (or vice versa) — after this, `find(a) == find(b) == find(c)`, call the shared root `c`. - Account "Mary": email `m`. No union needed (single email) — `m` is its own root. - Bucket pass: `{a, b, c}` all resolve to root `c` -> sorted set `{a, b, c}` -> label with `owner[c] = "John"`. `{m}` resolves to

root m -> label with owner[m] = "Mary" . - Result: ["John", a, b, c] and ["Mary", m] — matches the expected output.

Brute force

Repeatedly scan pairs of accounts and merge any that share an email until nothing changes — $O(\text{accounts}^2 \cdot \text{emails})$, clumsy and error-prone.

```
public List<List<String>> accountsMergeBrute(List<List<String>> accounts) {
    List<Set<String>> groups = new ArrayList<>();
    List<String> names = new ArrayList<>();
    for (List<String> acc : accounts) {
        Set<String> emails = new HashSet<>(acc.subList(1, acc.size()));
        int merged = -1;
        for (int i = 0; i < groups.size(); i++) {
            if (!Collections.disjoint(groups.get(i), emails)) { // shares an email
                groups.get(i).addAll(emails);
                merged = i;
                break;
            }
        }
        if (merged == -1) { groups.add(emails); names.add(acc.get(0)); }
    }
    // NOTE: a single pass can miss chains; a correct brute force loops until stable.
    List<List<String>> result = new ArrayList<>();
    for (int i = 0; i < groups.size(); i++) {
        List<String> sorted = new ArrayList<>(groups.get(i));
        Collections.sort(sorted);
        sorted.add(0, names.get(i));
        result.add(sorted);
    }
    return result;
}
```

Optimized (Union-Find on emails)

Union emails within each account; bucket by root; sort each bucket.

```

public List<List<String>> accountsMerge(List<List<String>> accounts) {
    Map<String, String> parent = new HashMap<>(); // email -> parent email
    Map<String, String> owner = new HashMap<>(); // email -> account name
    for (List<String> acc : accounts) {
        String name = acc.get(0);
        for (int i = 1; i < acc.size(); i++) {
            String email = acc.get(i);
            parent.putIfAbsent(email, email); // each email starts as its own root
            owner.put(email, name);
            if (i > 1) union(parent, acc.get(1), email); // link all emails in this account
        }
    }
    Map<String, TreeSet<String>> groups = new HashMap<>(); // root -> sorted emails
    for (String email : parent.keySet()) {
        String root = find(parent, email);
        groups.computeIfAbsent(root, x -> new TreeSet<>()).add(email);
    }
    List<List<String>> result = new ArrayList<>();
    for (Map.Entry<String, TreeSet<String>> e : groups.entrySet()) {
        List<String> merged = new ArrayList<>(e.getValue());
        merged.add(0, owner.get(e.getKey())); // prepend the account name
        result.add(merged);
    }
    return result;
}

private String find(Map<String, String> parent, String x) {
    while (!x.equals(parent.get(x))) {
        parent.put(x, parent.get(parent.get(x))); // path compression
        x = parent.get(x);
    }
    return x;
}

private void union(Map<String, String> parent, String a, String b) {
    parent.put(find(parent, a), find(parent, b));
}

```

Version	Time	Space
Pairwise merge	$O(\text{accounts}^2 \cdot \text{emails})$	$O(\text{emails})$
Union-Find	$O(E \cdot \alpha(E)) + \text{sorting}$	$O(\text{emails})$

5.2 Course Schedule II (LC 210)

Problem. You must take `numCourses` courses, labeled `0` to `numCourses - 1`. You are given `prerequisites`, an array of pairs `[a, b]` meaning course `b` must be completed **before** course `a`. Return **any one valid ordering** in which you could take all the courses. If it is impossible to finish all courses (the prerequisite graph has a cycle), return an empty array.

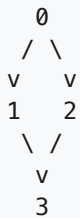
- **Constraints (typical LeetCode bounds):** $1 \leq \text{numCourses} \leq 2000$; $0 \leq \text{prerequisites.length} \leq \text{numCourses} * (\text{numCourses} - 1)$; no duplicate prerequisite pairs and no `[a, a]` self-prerequisite.

- **Clarifying assumptions:** any valid topological order is acceptable (the problem doesn't ask for the lexicographically smallest one); a course with no prerequisites at all can be taken at any point once its dependencies (zero of them) are satisfied, i.e. immediately.
- **Why it's tricky:** the real question hiding inside "find an ordering" is "detect a cycle," and a naive repeated-scan approach re-derives "what's ready" from scratch every round — Kahn's algorithm instead maintains in-degree counts incrementally so each edge is examined once.

Example.

```
Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
Output: [0, 1, 2, 3]      (0 first; 3 last)
```

Intuition (diagram — topological sort via Kahn's BFS). Draw the dependency DAG with an arrow $b \rightarrow a$ meaning "b unlocks a". A course with **in-degree 0** has no unmet prerequisites, so it's safe to take now. Take it, remove its outgoing arrows (decrement neighbors' in-degrees), and any neighbor that drops to 0 becomes newly available. If you can schedule all courses this way, there's no cycle.



Approach (step by step)

Key idea / invariant. A node may be dequeued (taken) only once **every** prerequisite edge pointing into it has already been "removed" — tracked by its `indegree` counter reaching `0`. The queue's invariant is: *at any moment, the queue holds exactly the courses whose prerequisites are all already scheduled*. If the algorithm finishes having scheduled fewer than `numCourses` courses, the remaining ones are stuck behind each other in a cycle.

1. Build the adjacency list `adj[b] = [a, ...]` (b unlocks a) and an `indegree[]` array by scanning every prerequisite pair once.
2. Seed a queue with every course whose `indegree == 0` — these need nothing and can go first.
3. Repeatedly pop a course from the queue, append it to `order`, then for each course it unlocks, decrement that course's in-degree. Any course whose in-degree just hit `0` becomes newly ready — push it onto the queue.
4. Stop when the queue is empty. If `order` contains all `numCourses` courses, return it; otherwise a cycle blocked some courses forever, so return `[]`.

Worked trace on the example (`prerequisites = [[1,0],[2,0],[3,1],[3,2]]`, so `0` unlocks `1` and `2`, and both `1` and `2` unlock `3`):

step	in-degrees (0,1,2,3)	queue	order so far
start	(0, 1, 1, 2)	[0]	[]
take 0, unlock 1,2	(0, 0, 0, 2)	[1, 2]	[0]
take 1, unlock 3	(0, 0, 0, 1)	[2]	[0, 1]
take 2, unlock 3	(0, 0, 0, 0)	[3]	[0, 1, 2]
take 3	(0, 0, 0, 0)	[]	[0, 1, 2, 3]

All 4 courses scheduled -> no cycle -> return [0, 1, 2, 3] .

Brute force

Repeatedly scan for any course whose prerequisites are all already placed; append it. Works but re-scans everything each round — $O(V^2 + VE)$.

```
public int[] findOrderBrute(int numCourses, int[][] prerequisites) {
    boolean[] taken = new boolean[numCourses];
    int[] order = new int[numCourses];
    int idx = 0;
    for (int round = 0; round < numCourses; round++) {
        boolean progressed = false;
        for (int course = 0; course < numCourses; course++) {
            if (taken[course]) continue;
            boolean ready = true;
            for (int[] p : prerequisites)
                if (p[0] == course && !taken[p[1]]) { ready = false; break; }
            if (ready) { taken[course] = true; order[idx++] = course; progressed = true; }
        }
        if (!progressed) break; // stuck -> cycle
    }
    return idx == numCourses ? order : new int[0];
}
```

Optimized (Kahn's algorithm — in-degree BFS)

Build adjacency + in-degrees once; peel off zero-in-degree nodes.


```

public int[] findOrder(int numCourses, int[][] prerequisites) {
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>());
    int[] indegree = new int[numCourses];
    for (int[] p : prerequisites) {
        adj.get(p[1]).add(p[0]);          // b -> a: b unlocks a
        indegree[p[0]]++;
    }
    Queue<Integer> ready = new LinkedList<>();
    for (int c = 0; c < numCourses; c++) if (indegree[c] == 0) ready.add(c);

    int[] order = new int[numCourses];
    int idx = 0;
    while (!ready.isEmpty()) {
        int course = ready.poll();
        order[idx++] = course;
        for (int next : adj.get(course)) {
            if (--indegree[next] == 0) ready.add(next);    // all prereqs satisfied
        }
    }
    return idx == numCourses ? order : new int[0];        // fewer than all -> cycle
}

```

Version	Time	Space
Repeated scan	$O(V^2 + V \cdot E)$	$O(V)$
Kahn's BFS	$O(V + E)$	$O(V + E)$

5.3 01 Matrix (LC 542)

Problem. Given an $m \times n$ binary matrix `mat` containing only 0s and 1s, return a matrix of the same size where each cell holds the **distance to the nearest 0** in `mat`, measured as the minimum number of 4-directional (up/down/left/right) steps.

- **Constraints (typical LeetCode bounds):** $1 \leq m, n \leq 10^4$, $1 \leq m * n \leq 10^4$; there is **at least one 0** in the matrix (so every cell has a finite answer).
- **Clarifying assumptions:** distance is measured in grid steps (Manhattan-style along the grid, not straight-line); a cell that is itself 0 has distance 0.
- **Why it's tricky:** the natural-looking approach — BFS from each 1 to find its nearest 0 — redoes overlapping work for every cell. The fix is to reverse the direction of the search: BFS outward from **all** the zeros simultaneously, so every cell is visited exactly once, by whichever zero reaches it first (which is automatically its nearest one).

Example.

Input:	Output:
0 0 0	0 0 0
0 1 0	0 1 0
1 1 1	1 2 1

Intuition (diagram — multi-source BFS). Instead of running a BFS from every 1 (slow), run **one** BFS starting from **all the 0s at once**. Seed the queue with every zero at distance 0; the wavefront expands outward one layer per BFS round, and the first time it reaches a cell is that cell's shortest distance.

layer 0 (the zeros, dist=0):	layer 1 (dist=1):	layer 2 (dist=2):
0 0 0	0 0 0	0 0 0
0 . 0	0 1 0	0 1 0
. . .	1 . 1	1 2 1

Approach (step by step)

Key idea / invariant. In a plain BFS, the queue invariant is "the queue holds all nodes at the current distance d , and processing them discovers all unvisited nodes at distance $d + 1$." Multi-source BFS keeps exactly the same invariant — the only change is that **all** zero-cells start in the queue at distance 0 instead of a single source. Because BFS always expands the nearest unvisited frontier first, the very first time any cell is reached, it is being reached by the closest zero — so a single pass correctly computes every cell's distance to its nearest zero, not just to whichever zero happens to be scanned first.

1. Scan the whole grid once: push every cell where `mat[r][c] == 0` into the queue with `dist[r][c] = 0`; mark every 1-cell as unvisited (`dist = -1`, a sentinel).
2. Run standard BFS: pop a cell, look at its 4 neighbors, and for any unvisited neighbor (`dist == -1`) set `dist[neighbor] = dist[current] + 1` and push it.
3. Because a cell is only pushed once (right when it transitions from `-1` to a real distance), each cell is processed exactly once — total work is $O(\text{rows} * \text{cols})$.
4. When the queue empties, `dist` holds every cell's distance to its nearest 0.

Worked trace on the example: the queue starts with every 0 cell — $(0,0)$, $(0,1)$, $(0,2)$, $(1,0)$ — each at distance 0. Popping $(0,1)$ reaches $(1,1)$ (the only 1 adjacent to a zero directly above) and sets `dist[1][1] = 1`. Popping $(1,0)$ reaches $(2,0)$ and sets `dist[2][0] = 1`; popping $(0,2)$ similarly is blocked by boundaries but a later step from $(1,2)$'s BFS predecessor sets `dist[2][2] = 1`. Finally $(2,1)$ is only reachable through already-visited distance-1 cells like $(1,1)$ or $(2,0)$, so it settles at `dist[2][1] = 1 + 1 = 2`. Final distances: 1 2 1 on row 2 — matches the expected output.

Brute force

For each 1, BFS out to find the closest 0 — repeats work across cells.

```

public int[][] updateMatrixBrute(int[][] mat) {
    int rows = mat.length, cols = mat[0].length;
    int[][] dist = new int[rows][cols];
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (mat[r][c] == 0) continue;
            boolean[][] seen = new boolean[rows][cols];
            Queue<int[]> q = new LinkedList<>();
            q.add(new int[]{r, c, 0}); seen[r][c] = true;
            while (!q.isEmpty()) {
                int[] cur = q.poll();
                if (mat[cur[0]][cur[1]] == 0) { dist[r][c] = cur[2]; break; }
                for (int[] d : dirs) {
                    int nr = cur[0]+d[0], nc = cur[1]+d[1];
                    if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && !seen[nr][nc]) {
                        seen[nr][nc] = true; q.add(new int[]{nr, nc, cur[2]+1});
                    }
                }
            }
        }
    }
    return dist;
}

```

Optimized (multi-source BFS from all zeros)

One queue seeded with every zero; distances fill outward.

```

public int[][] updateMatrix(int[][] mat) {
    int rows = mat.length, cols = mat[0].length;
    int[][] dist = new int[rows][cols];
    Queue<int[]> queue = new LinkedList<>();
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (mat[r][c] == 0) queue.add(new int[]{r, c}); // all zeros are sources
            else dist[r][c] = -1; // -1 marks "not yet reached"
        }
    }
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    while (!queue.isEmpty()) {
        int[] cell = queue.poll();
        for (int[] d : dirs) {
            int nr = cell[0] + d[0], nc = cell[1] + d[1];
            if (nr >= 0 && nr < rows && nc >= 0 && nc < cols && dist[nr][nc] == -1) {
                dist[nr][nc] = dist[cell[0]][cell[1]] + 1; // first visit = shortest
                queue.add(new int[]{nr, nc});
            }
        }
    }
    return dist;
}

```

Version	Time	Space
BFS per cell	$O((rows \cdot cols)^2)$	$O(rows \cdot cols)$
Multi-source BFS	$O(rows \cdot cols)$	$O(rows \cdot cols)$

5.4 Longest Increasing Path in a Matrix (LC 329)

Problem. Given an $m \times n$ integer matrix, find the length of the longest path such that, for every consecutive pair of cells on the path, the value **strictly increases**. From each cell you may move only up, down, left, or right (not diagonally), and you may not revisit a cell. Return the length of the longest such path (counted in number of cells visited). (*Hard — DFS + memoization = DP on an implicit DAG.*)

- **Constraints (typical LeetCode bounds):** $1 \leq m, n \leq 200$; $0 \leq \text{matrix}[i][j] \leq 2^{31} - 1$.
- **Clarifying assumptions:** a single cell counts as a path of length 1; there is no requirement that the path be contiguous in value (only strictly increasing, gaps are fine, e.g. $2 \rightarrow 6 \rightarrow 9$ skips values).
- **Why it's tricky:** it looks like it needs cycle detection (since the grid graph has cycles if you ignore direction), but the "strictly increasing" rule already guarantees no path can ever revisit a cell — a path can never return to a smaller-or-equal value it came from. That means the "next strictly greater neighbor" relation is a DAG, which is exactly when memoized DFS (i.e., DP over a graph) is safe and correct.

Example.

```
Input:
9 9 4
6 6 8
2 1 1
Output: 4          (1 -> 2 -> 6 -> 9)
```

Intuition (diagram — memoized DFS / DP on a DAG). Because you can only step to a **strictly greater** value, no path can cycle — the "increasing" arrows form a DAG. Define $\text{best}[r][c]$ = longest increasing path *starting* at that cell. It's $1 + \max(\text{best of the strictly-larger neighbors})$, with base case $\text{best}[r][c] = 1$ when no neighbor is larger. Memoize so each cell is solved once; the final answer is the max $\text{best}[r][c]$ over all cells.

```
  9   9   4
  ^   ^
  6   6   8
  ^
2<--1   1
```

Reading the arrows: $6 \rightarrow 9$ (up), $8 \rightarrow 4$ (up, col 2), $2 \rightarrow 6$ (up, col 0), and $1 \rightarrow 2$ (left, row 2) — each arrow points from a smaller value to a strictly larger neighbor. Chaining $1 \rightarrow 2 \rightarrow 6 \rightarrow 9$ gives the longest path, length 4.

Approach (step by step)

Key idea / invariant (recurrence in words). $\text{best}[r][c]$ means "the longest strictly increasing path that starts at cell (r, c) and moves only to strictly larger neighbors." The recurrence: $\text{best}[r][c] = 1 + \max(\text{best}[nr][nc])$ for every neighbor (nr, nc) with $\text{matrix}[nr][nc] > \text{matrix}[r][c]$, or just 1 if no neighbor qualifies (base case — a local maximum has nowhere higher to go, so the best path from it is itself alone). Because edges only go from smaller to strictly larger values, there are no cycles, so this

recursion always terminates and each cell's value only ever needs to be computed once — that's what the memo table caches.

1. For every cell (r, c) in the grid, run a DFS that recursively computes $best[r][c]$.
2. Inside the DFS: if $memo[r][c]$ is already filled, return it immediately (already solved).
3. Otherwise, look at all 4 neighbors; for each one with a strictly greater value, recurse into it and take the max of $1 + best[neighbor]$ across those.
4. If no neighbor qualifies, $best[r][c] = 1$ (base case).
5. Store the result in $memo[r][c]$ before returning it — this guarantees the next time any other cell's DFS reaches (r, c) , it's an $O(1)$ lookup, not a re-exploration.
6. The overall answer is $\max(best[r][c])$ across every starting cell.

Worked trace on the example ($matrix = [[9,9,4],[6,6,8],[2,1,1]]$): - $best[2][1]$ (value 1): only neighbor with a larger value is $(2,0) = 2$. So $best[2][1] = 1 + best[2][0]$. - $best[2][0]$ (value 2): larger neighbor is $(1,0) = 6$. So $best[2][0] = 1 + best[1][0]$. - $best[1][0]$ (value 6): larger neighbor is $(0,0) = 9$. So $best[1][0] = 1 + best[0][0]$. - $best[0][0]$ (value 9): no larger neighbor (9 is a local max along this chain) $\rightarrow best[0][0] = 1$. - Unwinding: $best[1][0] = 2$, $best[2][0] = 3$, $best[2][1] = 4$. The chain $1 \rightarrow 2 \rightarrow 6 \rightarrow 9$ has length 4, matching the expected output.

Brute force

Plain DFS from every cell without memo — exponential re-exploration.

```
public int longestIncreasingPathBrute(int[][] matrix) {
    int rows = matrix.length, cols = matrix[0].length, best = 0;
    for (int r = 0; r < rows; r++)
        for (int c = 0; c < cols; c++)
            best = Math.max(best, dfs(matrix, r, c));
    return best;
}

private int dfs(int[][] m, int r, int c) {
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    int best = 1;
    for (int[] d : dirs) {
        int nr = r+d[0], nc = c+d[1];
        if (nr >= 0 && nr < m.length && nc >= 0 && nc < m[0].length && m[nr][nc] > m[r][c]) {
            best = Math.max(best, 1 + dfs(m, nr, nc));
        }
    }
    return best;
}
```

Optimized (DFS + memoization)

Cache each cell's answer; every cell computed exactly once.

```

public int longestIncreasingPath(int[][] matrix) {
    int rows = matrix.length, cols = matrix[0].length, best = 0;
    Integer[][] memo = new Integer[rows][cols];
    for (int r = 0; r < rows; r++)
        for (int c = 0; c < cols; c++)
            best = Math.max(best, dfs(matrix, r, c, memo));
    return best;
}

private int dfs(int[][] m, int r, int c, Integer[][] memo) {
    if (memo[r][c] != null) return memo[r][c]; // already solved this cell
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    int best = 1;
    for (int[] d : dirs) {
        int nr = r + d[0], nc = c + d[1];
        if (nr >= 0 && nr < m.length && nc >= 0 && nc < m[0].length
            && m[nr][nc] > m[r][c]) {
            best = Math.max(best, 1 + dfs(m, nr, nc, memo));
        }
    }
    return memo[r][c] = best;
}

```

Version	Time	Space
Plain DFS	$O(2^{(\text{rows} \cdot \text{cols})})$ worst	$O(\text{rows} \cdot \text{cols})$ stack
DFS + memo	$O(\text{rows} \cdot \text{cols})$	$O(\text{rows} \cdot \text{cols})$

5.5 Cheapest / Shortest Bridge — 0-1 BFS on a Grid

Problem (Minimum Obstacle Removal, LC 2290). You are given an $m \times n$ grid where each cell is `0` (empty, freely walkable) or `1` (an obstacle you may remove to pass through). Starting at cell `(0, 0)` and moving only 4-directionally, reach cell `(m-1, n-1)`. Return the **minimum number of obstacles you must remove** along the best possible path.

- **Constraints (typical LeetCode bounds):** $1 \leq m, n \leq 10^5$, $2 \leq m * n \leq 10^5$; `grid[i][j]` is `0` or `1`; `grid[0][0] == grid[m-1][n-1] == 0` (the endpoints are always passable).
- **Clarifying assumptions:** you may remove any number of obstacles anywhere on the grid (not just ones adjacent to the current path so far — removal is a one-time global action per cell, and "cost" only counts obstacles that lie *on* the path you actually take); passing through an empty cell costs nothing.
- **Why it's tricky:** this looks like a shortest-path-with-weighted-edges problem that needs Dijkstra's priority queue, but because every edge costs only `0` (entering an empty cell) or `1` (breaking an obstacle), a full $O(\log n)$ priority queue is overkill — a plain deque, processed the right way, gives the same guarantee in linear time (**0-1 BFS**).

Example.

```
Input:
0 1 1
1 1 0
1 1 0
Output: 2
```

Intuition (diagram — 0-1 BFS with a deque). Moving into an empty cell costs 0 ; moving into an obstacle costs 1 . When all edge weights are 0 or 1 , a **deque** replaces the priority queue: push **0-cost** moves to the **front** (they belong to the *current* distance layer, so process them before anything already waiting) and **1-cost** moves to the **back** (they belong to the *next* distance layer). This keeps the deque sorted by distance at all times, without ever needing to sort.

cost to ENTER a cell: empty (0) costs 0 , obstacle (1) costs 1

```
0 1 1
1 1 0
1 1 0
```

one shortest path: $(0,0) \rightarrow (1,0)[\text{obstacle},+1] \rightarrow (2,0)[\text{obstacle},+1] \rightarrow (2,1)[\text{obstacle},+1]$
the cheaper path removes exactly 2 obstacles total (see trace below)

Approach (step by step)

Key idea / invariant. The deque's invariant is: *at every moment, the deque's contents are sorted by distance, front to back, with at most two distinct distance values present at once.* This holds because every push either goes to the front (same distance as the cell we popped) or the back (one more than the cell we popped) — so distances in the deque never jump around; they only ever increase by 0 or 1 as you move toward the back. Popping from the front therefore always gives the next-closest unvisited cell, exactly like Dijkstra's priority queue would, but without any log-factor bookkeeping.

1. Initialize `dist[0][0] = 0` (start is always empty per constraints) and push it to the front of the deque.
2. Pop from the **front** of the deque. For each of its 4 neighbors, compute `newDist = dist[current] + grid[neighbor]` (0 if the neighbor is empty, 1 if it's an obstacle).
3. If `newDist` improves `dist[neighbor]`, update it. Then push the neighbor to the **front** if the move cost 0 (it belongs to the same distance layer we're currently draining), or to the **back** if the move cost 1 (it belongs to the next layer, to be drained after the current one empties).
4. Repeat until the deque is empty. `dist[m-1][n-1]` is the answer.

Worked trace on the example (grid as shown, obstacles at $(0,1), (0,2), (1,0), (1,1), (2,0), (2,1)$): start $(0,0)$, `dist=0`, pushed front. Pop $(0,0)$: neighbor $(0,1)$ is an obstacle $\rightarrow$ `dist=1`, push back; neighbor $(1,0)$ is an obstacle $\rightarrow$ `dist=1`, push back. Deque now holds two distance-1 cells. Popping through them eventually reaches $(1,2)$ (empty, cost 0 from an obstacle neighbor already at distance 1) and $(2,2)$ (empty) at `dist=2` via a path that breaks exactly 2 obstacles — e.g. $(0,0) \rightarrow (1,0)[+1] \rightarrow (1,1)[+1] \rightarrow (1,2)[+0] \rightarrow (2,2)[+0]$. No path avoids breaking at least 2 obstacles, so the answer is 2 , matching expected output.

Brute force

Dijkstra with a priority queue also solves it (correct, just heavier: $O(E \log V)$).

```

public int minimumObstaclesBrute(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    int[][] dist = new int[rows][cols];
    for (int[] row : dist) Arrays.fill(row, Integer.MAX_VALUE);
    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b)->a[2]-b[2]); // {r,c,cost}
    dist[0][0] = grid[0][0];
    pq.add(new int[]{0,0,grid[0][0]});
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    while (!pq.isEmpty()) {
        int[] cur = pq.poll();
        if (cur[2] > dist[cur[0]][cur[1]]) continue;
        for (int[] d : dirs) {
            int nr=cur[0]+d[0], nc=cur[1]+d[1];
            if (nr>=0&&nr<rows&&nc>=0&&nc<cols) {
                int nd = cur[2] + grid[nr][nc];
                if (nd < dist[nr][nc]) { dist[nr][nc]=nd; pq.add(new int[]{nr,nc,nd}); }
            }
        }
    }
    return dist[rows-1][cols-1];
}

```

Optimized (0-1 BFS with a double-ended queue)

Front for free moves, back for costly ones — no log factor.

```

public int minimumObstacles(int[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    int[][] dist = new int[rows][cols];
    for (int[] row : dist) Arrays.fill(row, Integer.MAX_VALUE);
    Deque<int[]> deque = new ArrayDeque<>(); // {r, c}, ordered by distance
    dist[0][0] = 0;
    deque.offerFirst(new int[]{0, 0});
    int[][] dirs = {{1,0},{-1,0},{0,1},{0,-1}};
    while (!deque.isEmpty()) {
        int[] cell = deque.pollFirst();
        int r = cell[0], c = cell[1];
        for (int[] d : dirs) {
            int nr = r + d[0], nc = c + d[1];
            if (nr < 0 || nr >= rows || nc < 0 || nc >= cols) continue;
            int cost = grid[nr][nc]; // 0 to enter empty, 1 to break obstacle
            if (dist[r][c] + cost < dist[nr][nc]) {
                dist[nr][nc] = dist[r][c] + cost;
                if (cost == 0) deque.offerFirst(new int[]{nr, nc}); // free -> explore first
                else deque.offerLast(new int[]{nr, nc}); // costly -> later
            }
        }
    }
    return dist[rows - 1][cols - 1];
}

```

Version	Time	Space
Dijkstra (PQ)	$O(\text{cells} \cdot \log \text{cells})$	$O(\text{cells})$
0-1 BFS (deque)	$O(\text{cells})$	$O(\text{cells})$

5.6 Critical Connections in a Network (LC 1192)

Problem. There are n servers, numbered 0 to $n - 1$, connected by $n - 1$ or more undirected network cables given as `connections[i] = [a, b]`. The whole network is connected (you can reach any server from any other). A **critical connection** (a **bridge**) is a cable whose removal would leave at least two servers unable to reach each other. Return all critical connections, in any order. (*Hard — Tarjan's bridge-finding. Also the confirmed networking-flavored question from your Domain-Disguised volume; included here in its pure graph form.*)

- **Constraints (typical LeetCode bounds):** $2 \leq n \leq 10^5$; $n - 1 \leq \text{connections.length} \leq \min(10^5, n * (n - 1) / 2)$; no self-loops and no repeated edges between the same pair.
- **Clarifying assumptions:** the graph is guaranteed connected as given (no need to handle disconnected input); a "critical connection" is about edges, not nodes — you're never asked to find articulation points (cut vertices), only bridges.
- **Why it's tricky:** the brute-force instinct — remove each edge and re-check connectivity — is $O(E * (V + E))$ and doesn't generalize the insight. The real trick is that a **single** DFS pass can identify every bridge at once, using discovery times and "low-link" values, because a bridge is a purely *local* property of whether a subtree has any back edge escaping above its parent.

Example.

```
Input:  n = 4, connections = [[0,1],[1,2],[2,0],[1,3]]
Output: [[1,3]]           (the triangle 0-1-2 has redundancy; only 1-3 is a bridge)
```

Intuition (diagram — DFS discovery times + low-link). Run a DFS, stamping each node with a **discovery time** `disc` (the order nodes are first visited). For each node, track `low` = the smallest discovery time reachable from its DFS subtree by using zero or more tree edges plus **at most one** back edge. An edge (u, v) — where v is u 's DFS child — is a **bridge** exactly when `low[v] > disc[u]`: that inequality says v 's entire subtree has *no* back edge reaching u or anything discovered before u , so cutting (u, v) truly isolates that subtree.

```
0 --- 1 --- 3
 \   /
  2
```

The triangle $0-1-2$ is a cycle: every edge inside it has an alternate route around, so `low` can always "reach back" through the cycle — none of $0-1$, $1-2$, $2-0$ are bridges. The edge $1-3$ has no alternate route at all: cutting it isolates 3 completely — it is the bridge.

Approach (step by step)

Key idea / invariant. `disc[u]` is a timestamp: the order in which DFS first visits u . `low[u]` starts equal to `disc[u]` and can only ever get smaller as DFS explores u 's subtree — it tightens whenever a **back edge** (an edge to an already-visited ancestor, not the immediate parent) is found, pulling `low[u]` down to that ancestor's `disc` value. The invariant maintained throughout the DFS is: `low[u]` equals the smallest `disc` value reachable from u 's subtree using at most one edge that isn't a tree edge going strictly downward. After fully exploring a child v of u , if `low[v]` never became `<= disc[u]`, that

means nothing in `v`'s subtree can reach back to `u` or above except through the edge `(u, v)` itself — so removing it disconnects that subtree, i.e. it's a bridge.

1. Pick any start node (e.g. `0`) and run DFS, assigning each node an increasing `disc` value as it's first visited, and initializing `low[node] = disc[node]`.
2. For each neighbor of the current node `u`, skip the edge straight back to `u`'s own parent (that's not a back edge, it's the tree edge we arrived on).
3. If the neighbor `v` is unvisited, it's a new tree edge: recurse into `v`, then after that recursion returns, tighten `low[u] = min(low[u], low[v])` — `u` inherits any back-reach its child found. Then check the bridge condition: if `low[v] > disc[u]`, edge `(u, v)` is a bridge.
4. If the neighbor `v` is already visited (and isn't the parent), it's a back edge: tighten `low[u] = min(low[u], disc[v])` directly (no bridge check here — back edges are never bridges by definition, they're literally the "way around").
5. After the full DFS, `bridges` contains exactly the critical connections.

Worked trace on the example (`connections = [[0,1],[1,2],[2,0],[1,3]]`, DFS starts at `0`):

node	disc	low	reasoning
0	0	0	DFS root
1	1	0	back edge 2→0 (found via child 2) pulls low[1] down to disc[0]=0
2	2	0	back edge 2→0 directly: low[2] = min(low[2], disc[0]) = 0
3	3	3	no back edge from 3 anywhere → low[3] stays 3

Checking each tree edge on return: edge `(2, 0)` isn't a tree edge (it's the back edge just described). Edge `(1, 2)`: `low[2]=0` is not `> disc[1]=0`, so not a bridge. Edge `(0, 1)`: `low[1]=0` is not `> disc[0]=0`, so not a bridge. Edge `(1, 3)`: `low[3]=3 > disc[1]=1` — this **is** a bridge. Final answer: `[[1, 3]]`, matching the expected output.

Brute force

Remove each edge, run a connectivity check (DFS/union-find); if the graph splits, it's critical. $O(E \cdot (V+E))$.

```

public List<List<Integer>> criticalConnectionsBrute(int n, List<List<Integer>> connections)
    List<List<Integer>> result = new ArrayList<>();
    for (List<Integer> skip : connections) {
        Map<Integer, List<Integer>> adj = new HashMap<>();
        for (List<Integer> e : connections) {
            if (e == skip) continue; // pretend this edge is gone
            adj.computeIfAbsent(e.get(0), x -> new ArrayList<>()).add(e.get(1));
            adj.computeIfAbsent(e.get(1), x -> new ArrayList<>()).add(e.get(0));
        }
        boolean[] seen = new boolean[n];
        dfsCount(0, adj, seen);
        int visited = 0;
        for (boolean b : seen) if (b) visited++;
        if (visited < n) result.add(skip); // graph split -> bridge
    }
    return result;
}

private void dfsCount(int node, Map<Integer, List<Integer>> adj, boolean[] seen) {
    seen[node] = true;
    for (int nb : adj.getOrDefault(node, Collections.emptyList()))
        if (!seen[nb]) dfsCount(nb, adj, seen);
}

```

Optimized (Tarjan's bridges — one DFS)

Track discovery and low-link values; a bridge is where a child can't reach back.

```

public List<List<Integer>> criticalConnections(int n, List<List<Integer>> connections) {
    List<List<Integer>> adj = new ArrayList<>();
    for (int i = 0; i < n; i++) adj.add(new ArrayList<>());
    for (List<Integer> e : connections) {
        adj.get(e.get(0)).add(e.get(1));
        adj.get(e.get(1)).add(e.get(0));
    }
    int[] disc = new int[n]; // discovery time; 0 means unvisited
    int[] low = new int[n]; // earliest reachable discovery time
    List<List<Integer>> bridges = new ArrayList<>();
    dfs(0, -1, new int[]{1}, disc, low, adj, bridges);
    return bridges;
}

private void dfs(int node, int parent, int[] timer, int[] disc, int[] low,
    List<List<Integer>> adj, List<List<Integer>> bridges) {
    disc[node] = low[node] = timer[0]++;
    for (int next : adj.get(node)) {
        if (next == parent) continue; // don't go straight back up the tree
        if (disc[next] == 0) { // unvisited -> tree edge
            dfs(next, node, timer, disc, low, adj, bridges);
            low[node] = Math.min(low[node], low[next]);
            if (low[next] > disc[node]) { // no back route around (node,next)
                bridges.add(Arrays.asList(node, next));
            }
        } else { // back edge -> tighten low
            low[node] = Math.min(low[node], disc[next]);
        }
    }
}

```

Version	Time	Space
Remove-edge + connectivity	$O(E \cdot (V + E))$	$O(V + E)$
Tarjan's bridges	$O(V + E)$	$O(V + E)$

Closing notes

You've now got the advanced layer on top of Vol 1's twelve patterns:

- **Prefix sums / difference arrays** turn "many range queries/updates" into $O(1)$ each — the hash-map-of-prefixes trick (§1.1–1.3) is one of the highest-ROI recognitions in interviews.
- **Bit & math** tricks (XOR cancellation, $n \& (n-1)$, fast power, Euclid) are $O(1)/O(\log n)$ shortcuts that look like magic until you've drawn the bit columns once.
- **Monotonic stack/deque** is the answer whenever you hear "next greater", "span", "window max", or "largest rectangle" — each element in and out once, so $O(n)$.
- **Advanced DP** is about *naming the state*: interval $[i][j]$, two-string $[i][j]$, or a tiny state machine (hold/sold/rest). Write the recurrence in English first.
- **Advanced graphs**: reach for **union-find** on connectivity, **Kahn/DFS-colors** on ordering & cycles, **multi-source BFS** when there are many starts, **0-1 BFS** when edges cost 0 or 1, and **memoized DFS** for "longest path in a DAG".

The one habit that ties it all together: before you write a line, draw the picture — the prefix line, the bit columns, the stack, the DP table, or the graph. If you can draw the state transition, you can code it. That's why every problem in this volume showed the diagram first.